PRÁCTICA 1 - CIFRADO

Parte A: Cifrado simétrico con OpenSSL

Se usará la orden *enc* que sirve para cifrar/descifrar mensajes usando alguno de los algoritmos soportados. Para comprobar la colección de algoritmos soportados usaremos la orden *ciphers*.

A1.- Cifrar un mensaje de prueba

- **Crear un archivo de texto con un editor (elegir el editor dependiendo del entorno de trabajo).**

Esta primera parte de la práctica se ha realizado en un equipo Linux.

Se ha creado un archivo llamado Ej-A1.txt y se ha guardado en la ubicación ~/Escritorio/SSI/Cifrado. Su contenido es el mostrado en la siguiente imagen.



- **Cifrar el fichero utilizando un algoritmo simétrico. Dejar el resultado en otro fichero.**

Para encriptar de forma que se usen las rutinas de cifrado simétrico se utiliza enc y se establece el método de cifrado –des en este caso, para indicar un fichero de entrada se usa –in y el de salida con –out, ambos con la ruta completa al archivo. Con -k 123 indicamos que la clave necesaria para cifrar y descifrar ese mensaje es la cadena “123”; de esta

forma, se puede hacer el proceso automático sin tener que introducir la contraseña a mano desde la terminal.

Abrimos la consola de OpenSSL a través del comando “openssl” y para encriptar el archivo introduciremos el siguiente comando:

enc -des -in ~/Escritorio/SSI/Cifrado/EjA1.txt -out ~/Escritorio/SSI/Cifrado/SalidaA1.txt

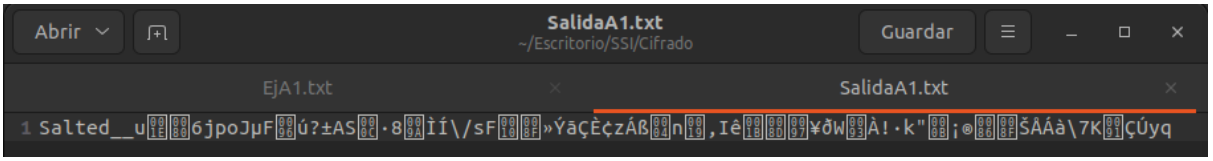
Sin usar -k pedirá la contraseña de encriptado por pantalla, pero no se mostrará como forma de seguridad:

```
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$ openssl enc -des -in ~/Escritorio/SSI/Cifrado/EjA1.txt -out
 ~/Escritorio/SSI/Cifrado/SalidaA1.txt
enter des-cbc encryption password:
Verifying - enter des-cbc encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$
```

Con -k no habrá realimentación de ningún tipo:

```
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$ openssl enc -des -k 123 -in ~/Escritorio/SSI/Cifrado/EjA1.t
xt -out ~/Escritorio/SSI/Cifrado/SalidaA1.txt
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$
```

Y ahora en la dirección ~/Escritorio/SSI/Cifrado podemos encontrar el archivo SalidaA1.txt con el siguiente aspecto:



Aparece al inicio del texto cifrado la palabra Salted__, lo que significa que cuando se vuelva a cifrar el mismo texto con la misma contraseña dará una salida diferente, pues depende de más parámetros. Para evitar esto se podría usar la opción -nosalt.

- **Realizar la transformación inversa (descifrar) a la realizada en el paso anterior.**

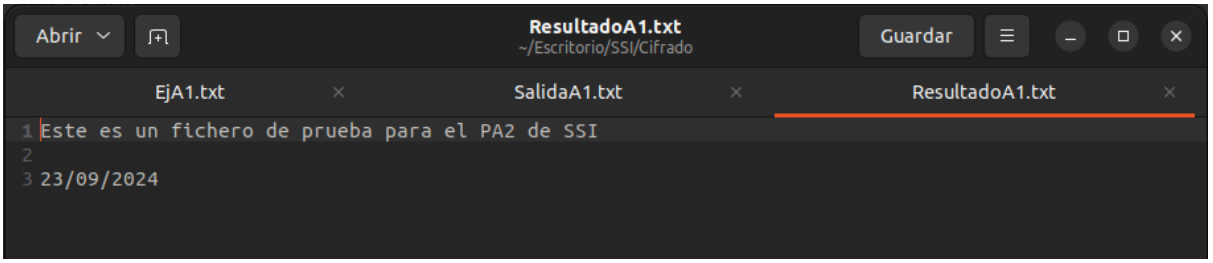
Para desencriptar, se usa la misma clave usada para encriptar (123) y se usa –d para indicar que la acción a realizar es la de descifrar, la –in será ahora la salida el anterior y como -out se indicará el nombre del fichero que queremos como salida del descifrado (resultadoA1.txt):

openssl enc –des –d –k 123 –in ~/Escritorio/SSI/Cifrado/SalidaA1.txt -out ~/Escritorio/SSI/Cifrado/ResultadoA1.txt

```
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$ openssl enc -des -d -k 123 -in ~/Escritorio/SSI/Cifrado/Sal
idaA1.txt -out ~/Escritorio/SSI/Cifrado/ResultadoA1.txt
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$
```

- **Comprobar que el mensaje obtenido en el proceso de descifrado es idéntico al original.**

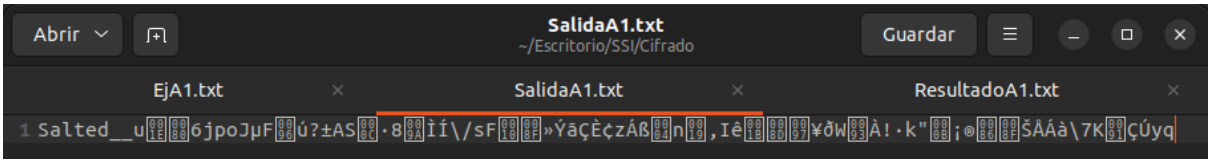
Como era de esperar, sí que se obtuvo el mismo mensaje:



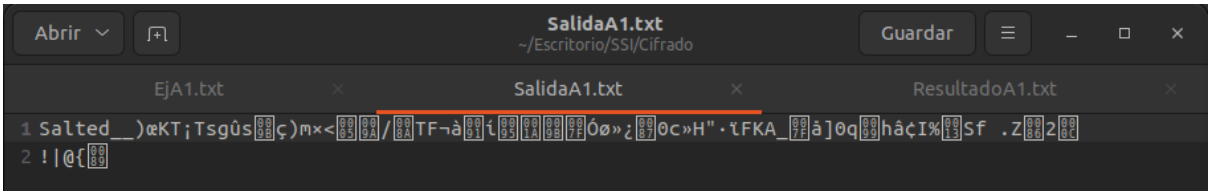
- **Repetir con variantes para diferentes algoritmos, claves, longitudes de mensajes, modos de operación y formas de introducir la clave de cifrado. contestar de forma razonada a las preguntas:**

Si en el ejemplo del ejercicio anterior, con Ej-A1.txt se cambia la clave a la hora de cifrarlo, se obtiene:

clave = 123



clave = 456

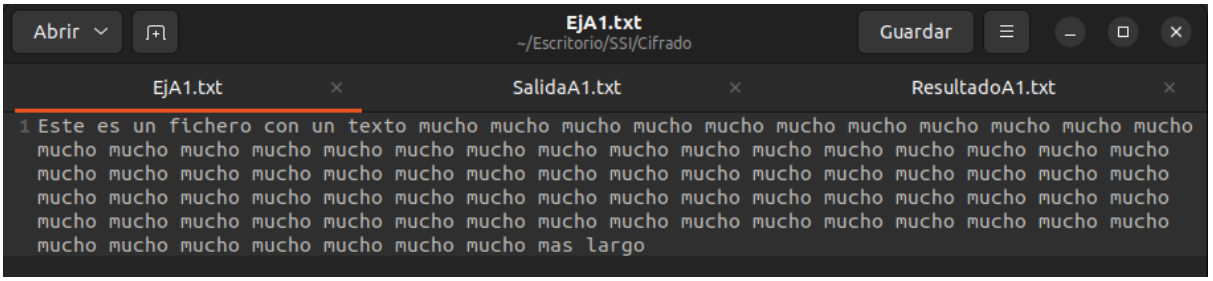


clave = 789

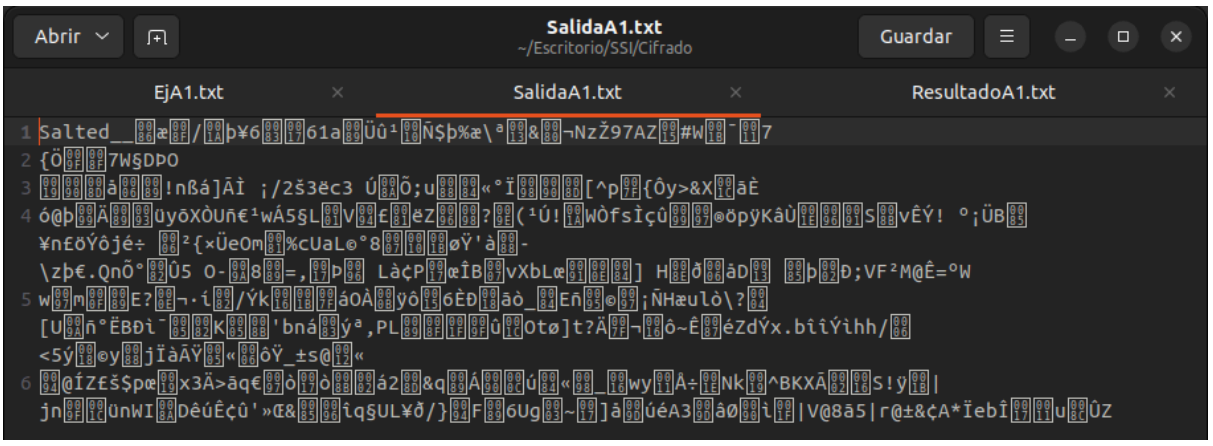


Cuando se vuelve a cifrar el mismo texto con la misma contraseña dará una salida diferente, pues depende de más parámetros. Para evitar esto se podría usar la opción -nosalt. Y al cifrarlo con una contraseña diferente, la salida es diferente puesto que la contraseña es un parámetro del cifrado.

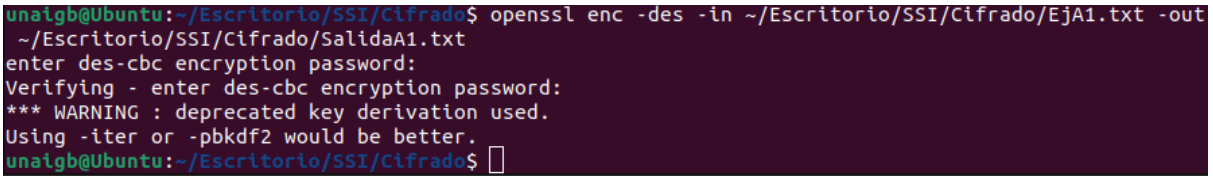
Si cambiaramos el contenido de EjA1.txt y lo aumentamos:

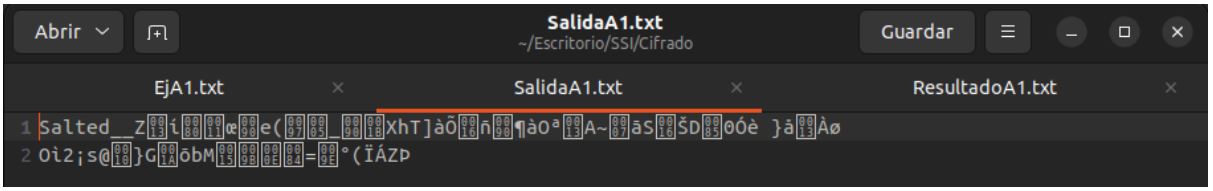


El texto cifrado sí que ha aumentado su longitud:



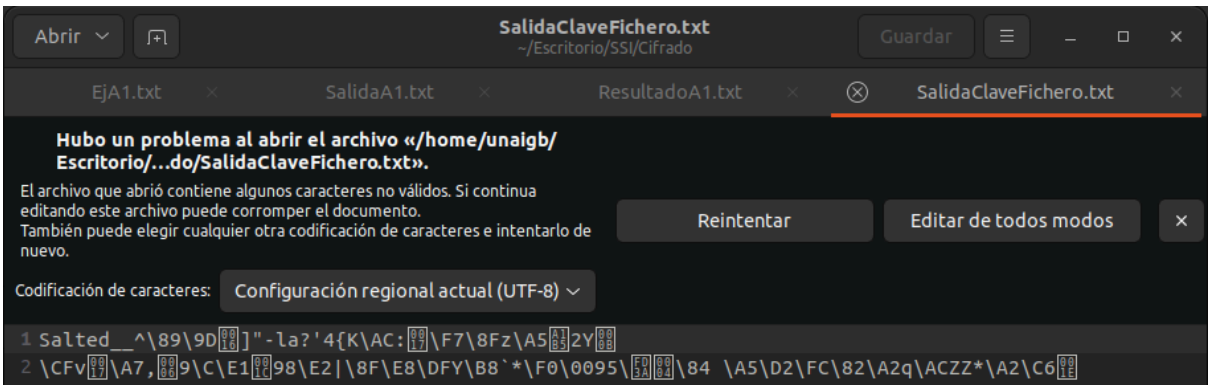
Si volvemos al texto original y en lugar de introducir la clave con -k, la introducimos de forma interactiva, el texto cifrado es el siguiente (usando el primer .txt para cifrar):





O si se introduce clave mediante un fichero:

openssl enc -des -kfile ~/Escritorio/SSI/Cifrado/EjA1.txt -in ~/Escritorio/SSI/Cifrado/EjA1.txt -out ~/Escritorio/SSI/Cifrado/SalidaClaveFichero.txt



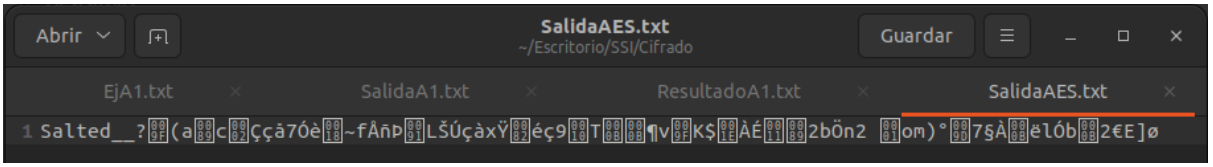
Se puede observar que la longitud es similar.

Si se utiliza en lugar de DES, el algoritmo AES:

Por ejemplo, aes-256-cbc:

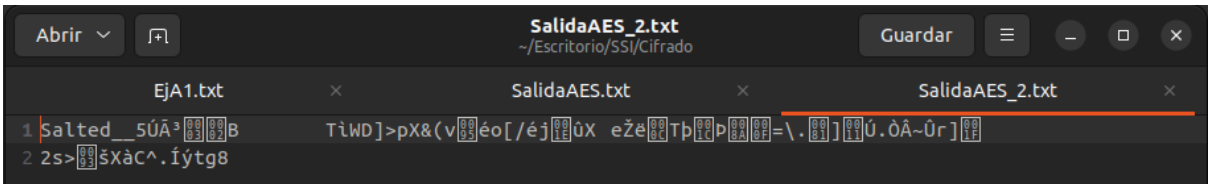
openssl enc -aes-256-cbc -k 123 -in ~/Escritorio/SSI/Cifrado/EjA1.txt -out ~/Escritorio/SSI/Cifrado/SalidaAES.txt

Se obtiene lo siguiente:



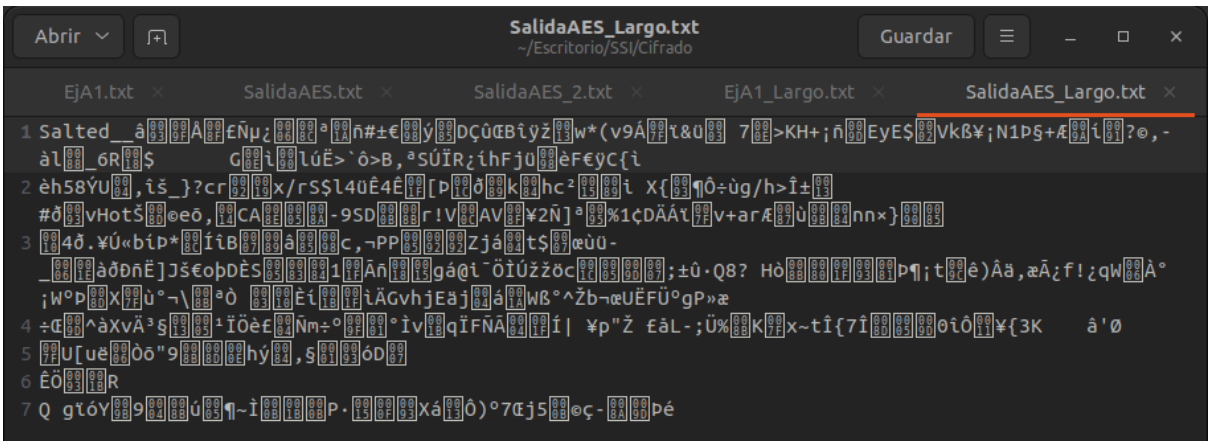
Si usando aes, cambiamos la clave por ejemplo por “clave”:

openssl enc -aes-256-cbc -k clave -in ~/Escritorio/SSI/Cifrado/EjA1.txt -out ~/Escritorio/SSI/Cifrado/SalidaAES_2.txt



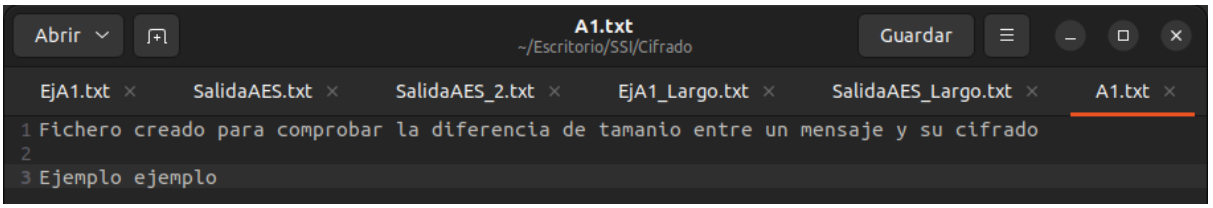
Y si usando este algoritmo, se cifra el .txt mucho más largo que utilizamos antes, se obtiene:

openssl enc -aes-256-cbc -k 123 -in ~/Escritorio/SSI/Cifrado/EjA1_Largo.txt -out ~/Escritorio/SSI/Cifrado/SalidaAES_Largo.txt



- **¿El mensaje cifrado es de mayor o menor tamaño que el mensaje original?**

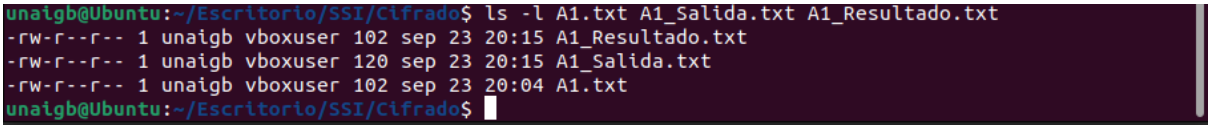
En cuanto a los tamaños de los archivos, tomando como referencia el siguiente archivo de ejemplo:



Tras cifrar y descifrar con las siguientes órdenes:

openssl enc –des –k 123 –in ~/Escritorio/SSI/Cifrado/A1.txt -out ~/Escritorio/SSI/Cifrado/A1_Salida.txt

openssl enc –des –d –k 123 –in ~/Escritorio/SSI/Cifrado/A1_Salida.txt -out ~/Escritorio/SSI/Cifrado/A1_Resultado.txt



Donde se puede ver que a la hora de cifrar el archivo, aumenta de tamaño pero al descifrarlo vuelve a su tamaño original. Y esto es debido a que se le añade información además de mezclarse la información con la clave de cifrado.

- **¿El tamaño depende del algoritmo de cifrado utilizado, del tamaño del mensaje original o de ambos?**

El tamaño del mensaje cifrado va a depender del contenido del mensaje original como hemos

visto. Y en cuanto a los algoritmos de cifrado, a continuación se muestra un ejemplo de cifrado del archivo anteriormente visto a1.txt con:

A1cifrado.txt ha sido cifrado en des.

A1cifrado2.txt ha sido cifrado en aes-256-cbc

A1cifrado3.txt ha sido cifrado en des-ecb

A1cifrado4.txt ha sido cifrado en rc4

A1cifrado5.txt ha sido cifrado en idea

```
-rw-r--r-- 1 unaigb vboxuser 128 sep 24 18:28 A1cifrado2.txt
-rw-r--r-- 1 unaigb vboxuser 120 sep 24 18:28 A1cifrado3.txt
-rw-r--r-- 1 unaigb vboxuser 118 sep 24 18:29 A1cifrado4.txt
-rw-r--r-- 1 unaigb vboxuser 120 sep 24 18:29 A1cifrado5.txt
-rw-r--r-- 1 unaigb vboxuser 120 sep 24 18:28 A1cifrado.txt
-rw-r--r-- 1 unaigb vboxuser 102 sep 23 20:04 A1.txt
```

Se puede comprobar que entre los archivos cifrados con distintos algoritmos hay una sutil

diferencia variando entre 102 y 128 bytes, por lo que el algoritmo en el que se cifra un fichero

de texto también influye un poco en el tamaño del fichero ya cifrado.

Probar y documentar las opciones de cifrado:
-in filename: Sirve para seleccionar el fichero de entrada, por ejemplo el que se va a cifrar en caso de que se vaya a cifrar o que se va a descifrar si se ha utilizado -d.
-out filename: Sirve para establecer el nombre del fichero de salida de la operación realizada, escribiendo su ruta completa si lo que se quiere es que se almacene en un directorio en específico.
-pass arg: En la versión que hemos empleado para realizar este informe, esta opción no está cumplimentada, como podemos ver si usamos openssl help

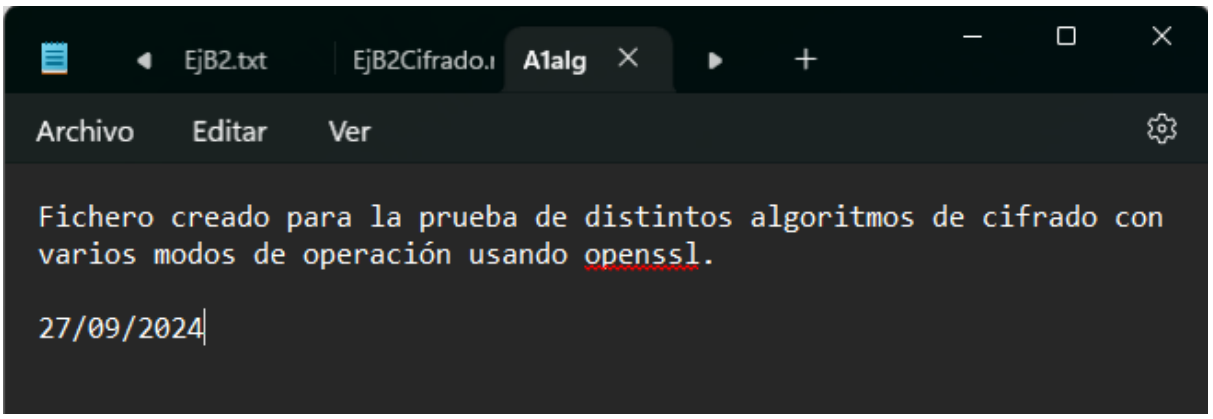
```
unaigb@Ubuntu:~/Escritorio/SSI/Cifrado$ openssl help
Standard commands
asn1parse         ca                ciphers           cms
crl               crl2pkcs7         dgst              dhparam
dsa               dsaparam          ec                ecparam
enc               engine            errstr            gendsa
genpkey           genrsa            help              list
nseq              ocsp              passwd            pkcs12
pkcs7             pkcs8             pkey              pkeyparam
pkeyutl           prime             rand              rehash
req               rsa               rsautl            s_client
s_server          s_time            sess_id           smime
speed             spkac             srp               storeutl
ts                verify            version           x509
```

En la versión que estamos utilizando, que es la 1.1.1 se puede usar passwd para generar contraseñas cifradas o hashes de contraseñas utilizando diferentes algoritmos de hash.
-salt: Está establecido por defecto. Se generará un salt aleatorio a la hora de cifrar, que provocará que el fichero cifrado con los mismos argumentos, salga diferente en cada momento. Se puede especificar el salt con la opción -s añadiendo a continuación un número hexadecimal en específico que usará como semilla, o desactivar esta aleatoriedad con la siguiente opción.
-nosalt: Evita que use números aleatorios, así el cifrado no contendrá la palabra “Salted”; y, el mismo fichero cifrado con los mismos argumentos varias veces proporcionará salidas cifradas idénticas.
-e: Sirve para encriptar los datos (encrypt), no hace falta ponerlo ya que es el que considera por defecto.
-d: Sirve para poder desencriptar un texto o fichero que ya se encuentra cifrado y se le pasará por ejemplo con -in.
-a: La encriptación tendrá lugar y después se codifica en base 64 y no en binario. En la desencriptación, primero se decodifica de base 64 y luego se desencripta.
-base64: Es igual que –a.
-k password: Sirve para poder introducir la clave mediante terminal en el propio comando en lugar de pedirla de forma activa al usuario.

-kfile filename: Lee y utiliza la clave de la primera línea del fichero que se indique en “filename”.

- **Probar con al menos 3 algoritmos de cifrado y con los 4 modos de operación básicos con cada uno de ellos.**

Usaremos el fichero A1algoritmos.txt para esta prueba:



ALGORITMO DES:

des-ecb:

openssl enc -des-ecb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1des-ecb.txt

openssl enc -des-ecb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1des-ecb.txt -out C:\Users\unaig\Desktop\SSI\A1des-ecbDescifrado.txt

des-cbc:

openssl enc -des-cbc -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1des-cbc.txt

openssl enc -des-cbc -d -k segura -in C:\Users\unaig\Desktop\SSI\A1des-cbc.txt -out C:\Users\unaig\Desktop\SSI\A1des-cbcDescifrado.txt

des-cfb:

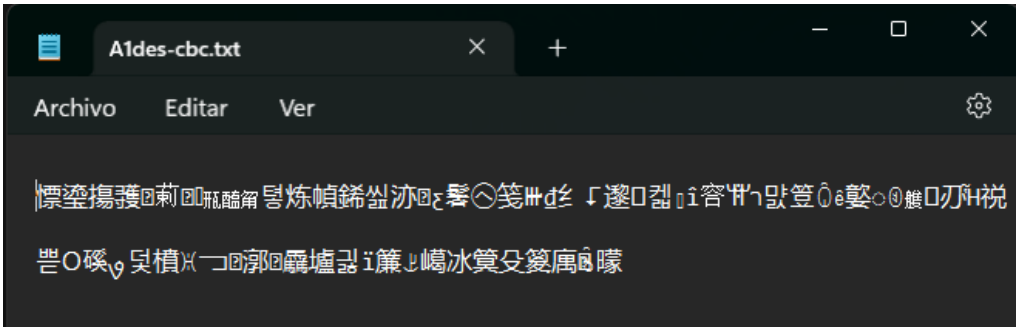
openssl enc -des-cfb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1des-cfb.txt

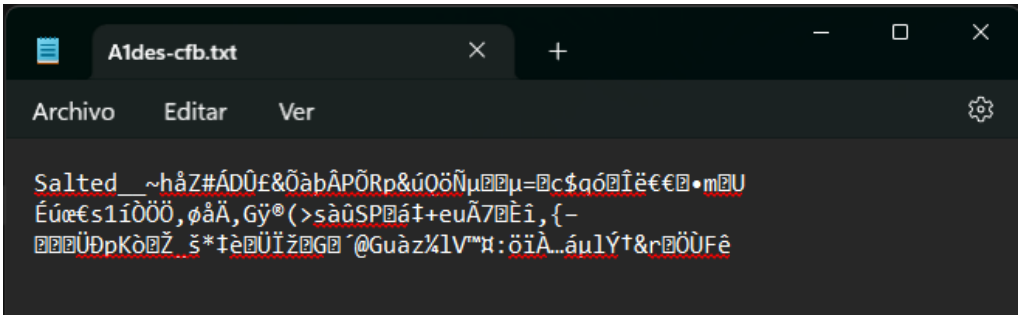
openssl enc -des-cfb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1des-cfb.txt -out C:\Users\unaig\Desktop\SSI\A1des-cfbDescifrado.txt

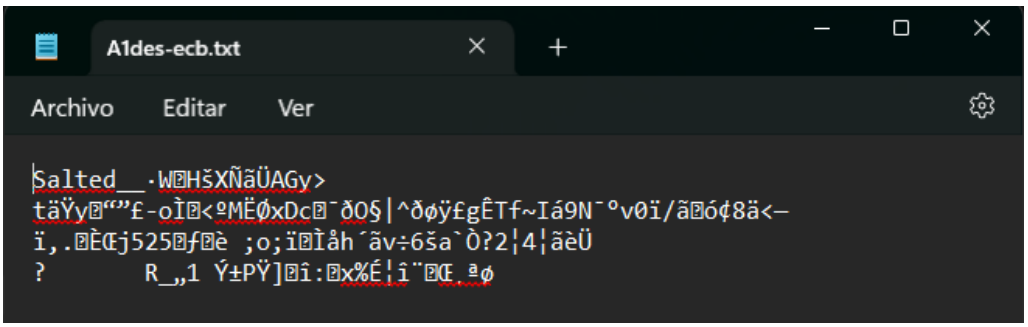
des-ofb:

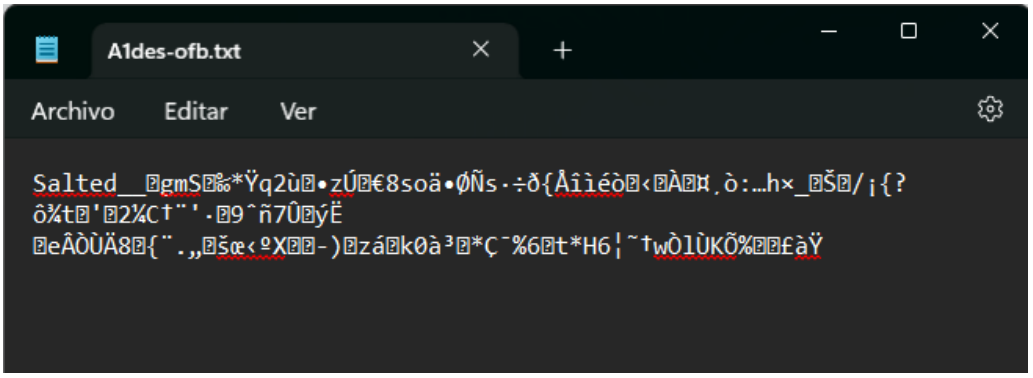
openssl enc -des-ofb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1des-ofb.txt

openssl enc -des-ofb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1des-ofb.txt -out C:\Users\unaig\Desktop\SSI\A1des-ofbDescifrado.txt









ALGORITMO AES:

aes-256-ecb:

openssl enc -aes-256-ecb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-ecb.txt

openssl enc -aes-256-ecb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1aes-256-ecb.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-ecbDescifrado.txt

aes-256-cbc:

openssl enc -aes-256-cbc -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-cbc.txt

openssl enc -aes-256-cbc -d -k segura -in C:\Users\unaig\Desktop\SSI\A1aes-256-cbc.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-cbcDescifrado.txt

aes-256-cfb:

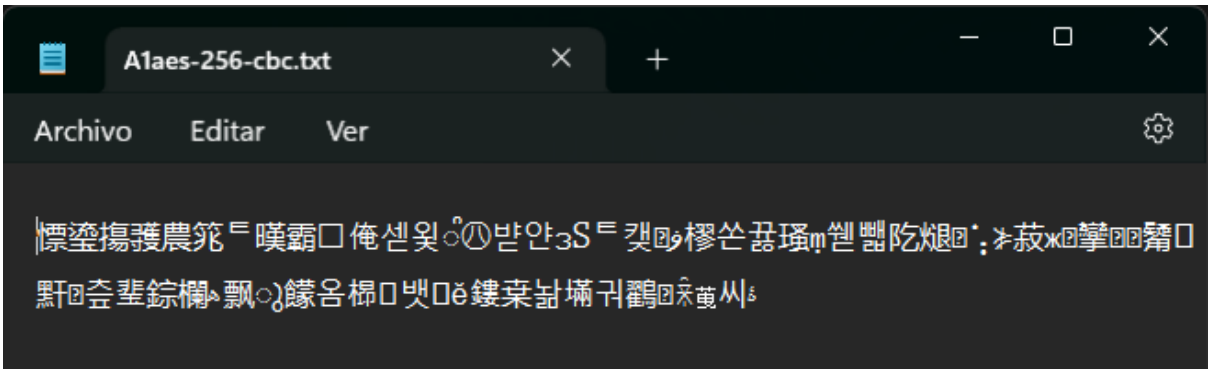
openssl enc -aes-256-cfb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-cfb.txt

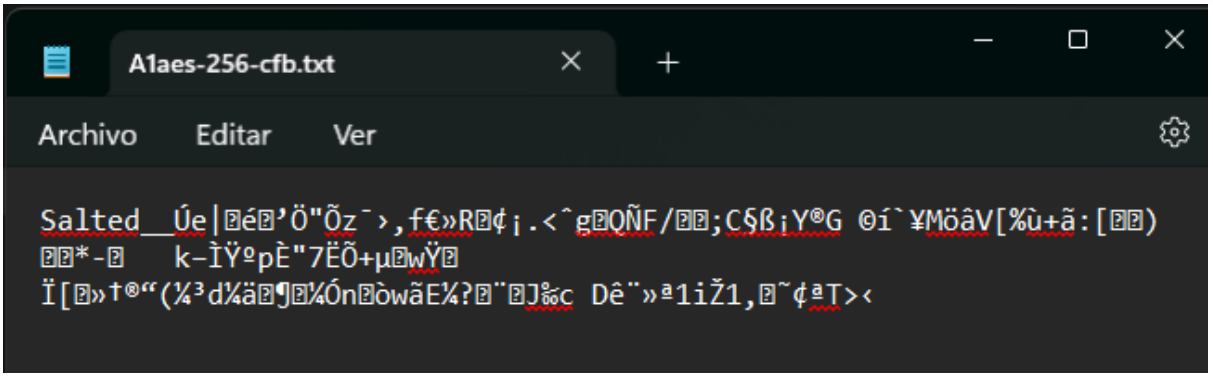
openssl enc -aes-256-cfb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1aes-256-cfb.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-cfbDescifrado.txt

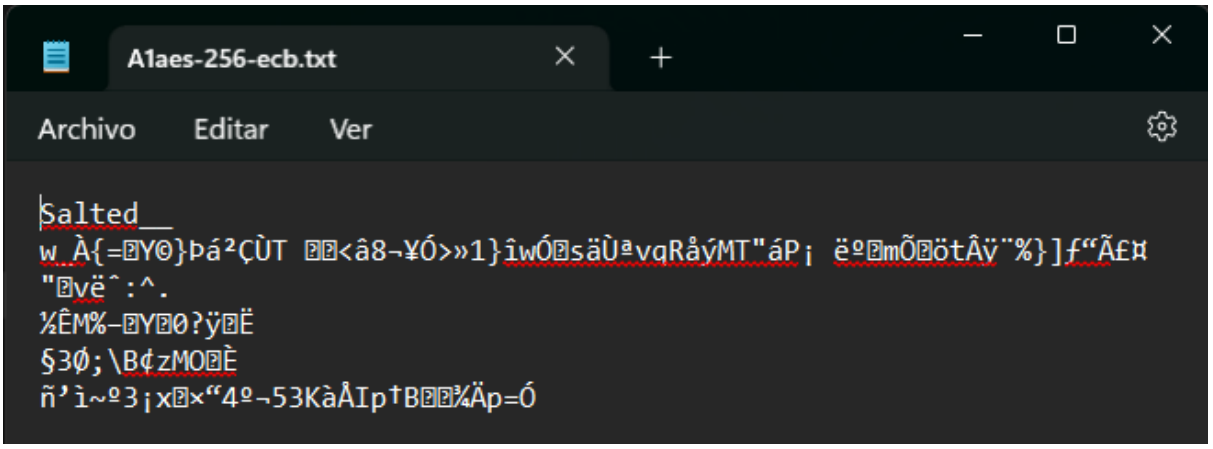
aes-256-ofb:

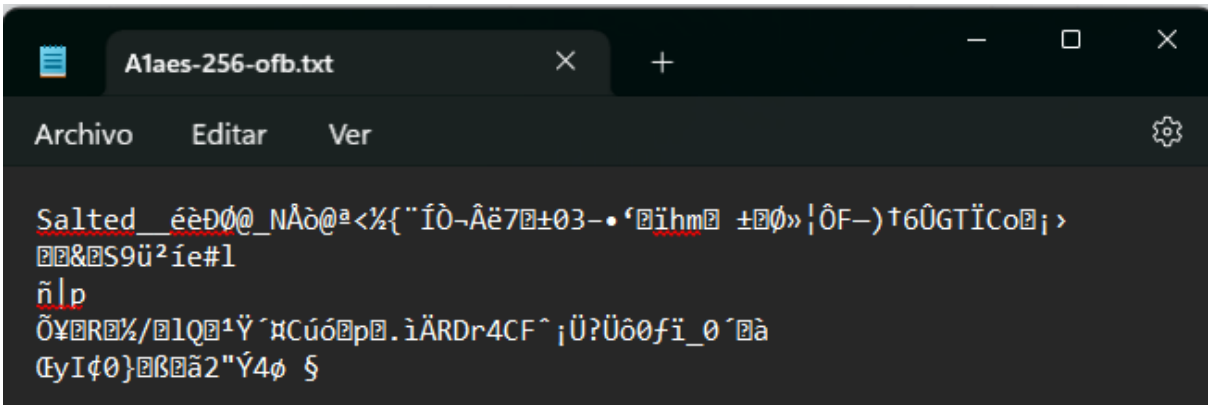
openssl enc -aes-256-ofb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-ofb.txt

openssl enc -aes-256-ofb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1aes-256-ofb.txt -out C:\Users\unaig\Desktop\SSI\A1aes-256-ofbDescifrado.txt









ALGORITMO RC2:

rc2-ecb:

openssl enc -rc2-ecb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-ecb.txt

openssl enc -rc2-ecb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1rc2-ecb.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-ecbDescifrado.txt

rc2-cbc:

openssl enc -rc2-cbc -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-cbc.txt

openssl enc -rc2-cbc -d -k segura -in C:\Users\unaig\Desktop\SSI\A1rc2-cbc.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-cbcDescifrado.txt

rc2-cfb:

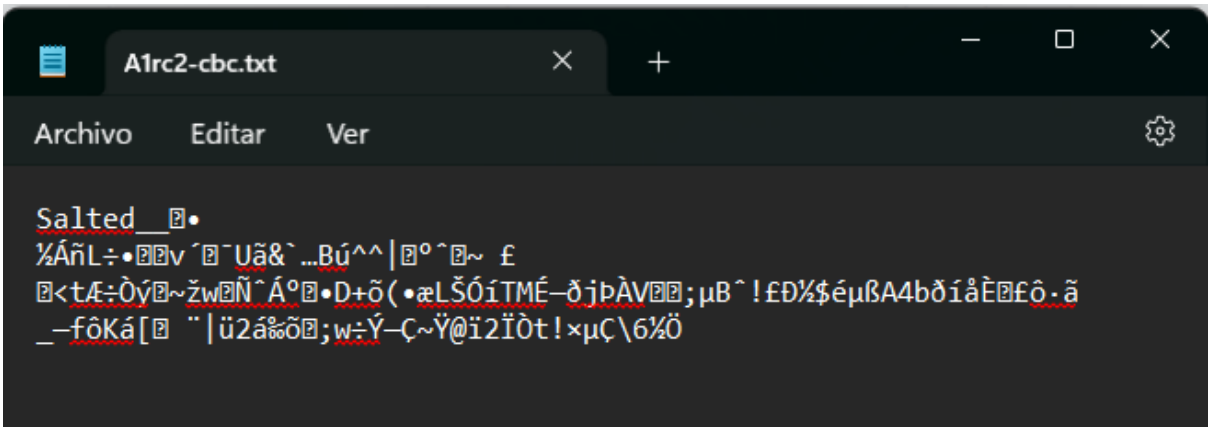
openssl enc -rc2-cfb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-cfb.txt

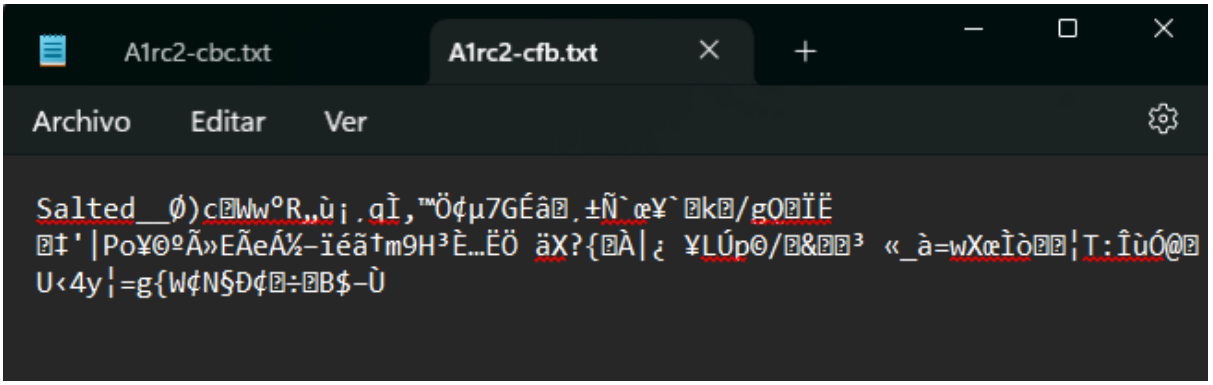
openssl enc -rc2-cfb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1rc2-cfb.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-cfbDescifrado.txt

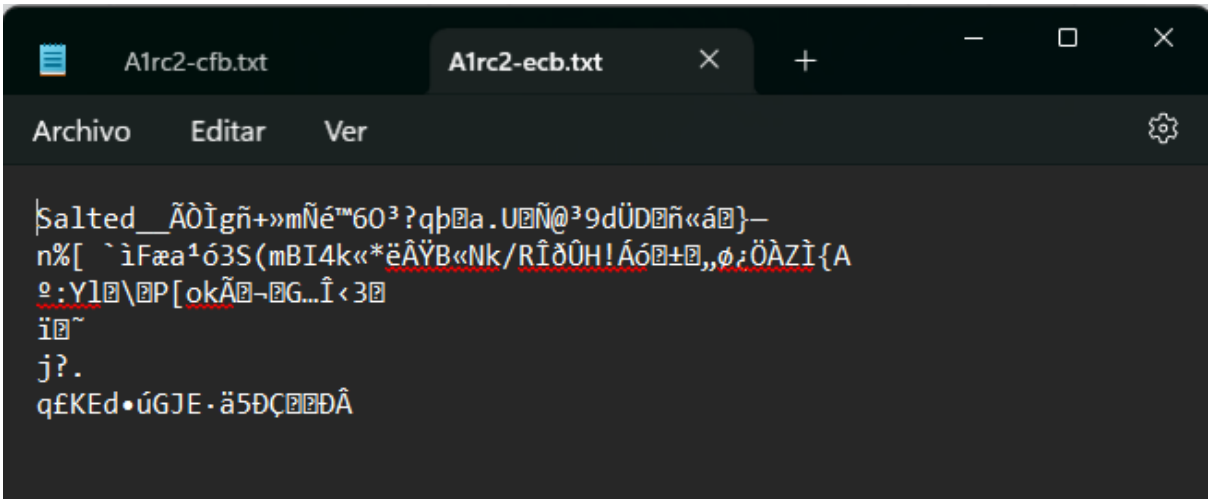
rc2-ofb:

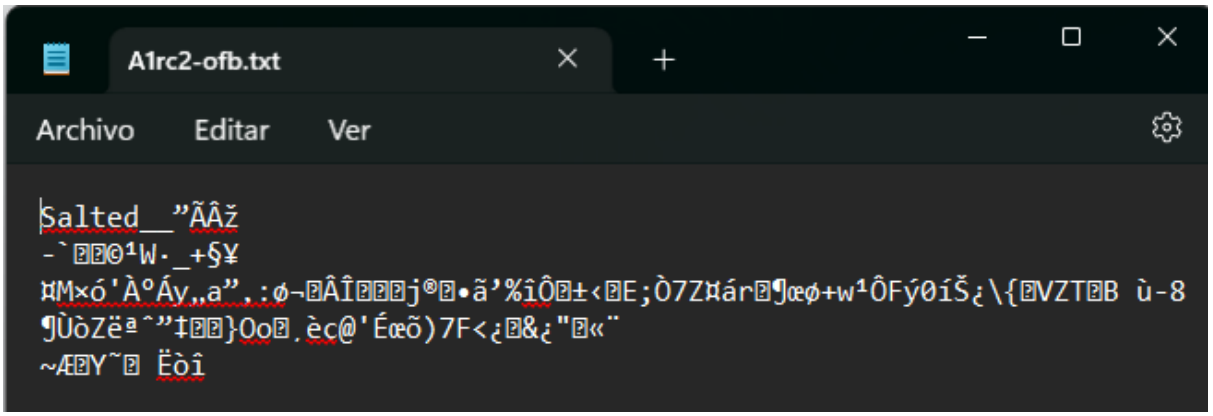
openssl enc -rc2-ofb -k segura -in C:\Users\unaig\Desktop\SSI\A1algoritmos.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-ofb.txt

openssl enc -rc2-ofb -d -k segura -in C:\Users\unaig\Desktop\SSI\A1rc2-ofb.txt -out C:\Users\unaig\Desktop\SSI\A1rc2-ofbDescifrado.txt

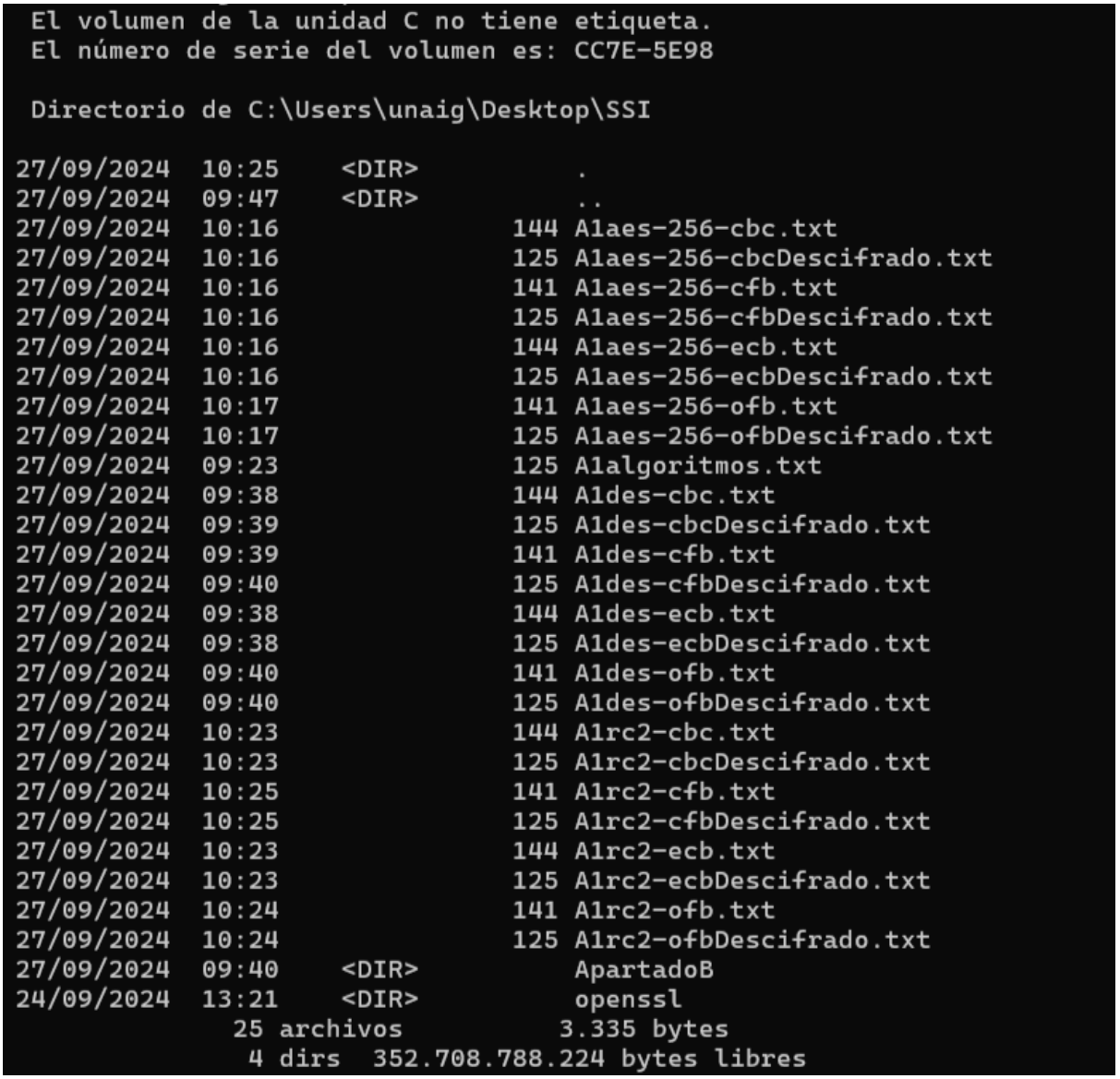

A1rc2-cbc.txt
Archivo
Editar
Ver
Salted__
½ÁñL÷•
v
Uã&`…Bú^^|
~ £
tÆ÷Òý
~žw
ÑÁº
D+õ(•æLŠÓíTMÉ–ðjÞÀV
;µB
!£Đ½$éµßA4bðíåÈ
£ô·ã
_–fôKá[
¨|ü2á‰õ
;w÷Ý–Ç~Ÿ@ï2ÏÒt!×µÇ\6%Ö


A1rc2-cbc.txt
A1rc2-cfb.txt
Archivo
Editar
Ver
Salted__Ø)c
Ww°R„ù¡.qÌ,™Ö¢µ7GÉâ
.±Ñ`œ¥`
k
/gQ
ÏË
‡'|Po¥©ºÃ»EÃeÁ¼–ïéã†m9H³È…ËÖ äX?{
À|¿ ¥LÚp©/
&
³ «_à=wXœÌò
¦T:ÎùÓ@
U‹4y¦=g{W¢N§Đ¢
÷
B$–Ù


A1rc2-cfb.txt
A1rc2-ecb.txt
Archivo
Editar
Ver
Salted__ÃÒÌgñ+»mÑé™6O³?qþ
a.U
Ñ@³9dÜD
ñ«á
}—
n%[`ìFæa¹ó3S(mBI4k«*ëÂŸB«Nk/RÎðÛH!Áó
±
„ø¿ÖÀZÌ{A
º:Yl
\
P[okÃ
¬
G…Î‹3
ï
˜
j?.
q£KEd•úGJE·ä5ÐÇ
ÐÂ


A1rc2-ofb.txt
Archivo
Editar
Ver
Salted__”ÃÂž
-`
O¹W·_+§¥
¤M×ó'ÀºÁy„a”.:ø¬
ÂÎ
j®
•ã’%îÔ
±‹
E;Ò7Z¤ár
¶œø+w¹ÔFý0íŠ¿\{
VZT
B ù-8
¶ÙòZë ª ˆ”‡
}Oo
.èç@'Éœõ)7F<¿
&¿"
«¨
~Æ
Y˜ Ëòî

Ahora haremos la comparación de tamaño entre todos los archivos generados:



```
 El volumen de la unidad C no tiene etiqueta.
 El número de serie del volumen es: CC7E-5E98

 Directorio de C:\Users\unaig\Desktop\SSI

27/09/2024  10:25    <DIR>          .
27/09/2024  09:47    <DIR>          ..
27/09/2024  10:16               144 A1aes-256-cbc.txt
27/09/2024  10:16               125 A1aes-256-cbcDescifrado.txt
27/09/2024  10:16               141 A1aes-256-cfb.txt
27/09/2024  10:16               125 A1aes-256-cfbDescifrado.txt
27/09/2024  10:16               144 A1aes-256-ecb.txt
27/09/2024  10:16               125 A1aes-256-ecbDescifrado.txt
27/09/2024  10:17               141 A1aes-256-ofb.txt
27/09/2024  10:17               125 A1aes-256-ofbDescifrado.txt
27/09/2024  09:23               125 A1algoritmos.txt
27/09/2024  09:38               144 A1des-cbc.txt
27/09/2024  09:39               125 A1des-cbcDescifrado.txt
27/09/2024  09:39               141 A1des-cfb.txt
27/09/2024  09:40               125 A1des-cfbDescifrado.txt
27/09/2024  09:38               144 A1des-ecb.txt
27/09/2024  09:38               125 A1des-ecbDescifrado.txt
27/09/2024  09:40               141 A1des-ofb.txt
27/09/2024  09:40               125 A1des-ofbDescifrado.txt
27/09/2024  10:23               144 A1rc2-cbc.txt
27/09/2024  10:23               125 A1rc2-cbcDescifrado.txt
27/09/2024  10:25               141 A1rc2-cfb.txt
27/09/2024  10:25               125 A1rc2-cfbDescifrado.txt
27/09/2024  10:23               144 A1rc2-ecb.txt
27/09/2024  10:23               125 A1rc2-ecbDescifrado.txt
27/09/2024  10:24               141 A1rc2-ofb.txt
27/09/2024  10:24               125 A1rc2-ofbDescifrado.txt
27/09/2024  09:40    <DIR>          ApartadoB
24/09/2024  13:21    <DIR>          openssl
              25 archivos          3.335 bytes
               4 dirs  352.708.788.224 bytes libres
```

En cuanto a los tamaños de los archivos se puede ver cómo los cifrados entre todos los algoritmos varían entre 141 y 144 bytes.

Es decir, dentro de los distintos algoritmos de cifrado, el tamaño final del texto cifrado varía según el modo de operación empleado, ocupando menos si se emplean los modos cfb y ofb que si se emplean los modos ecb y cbc.

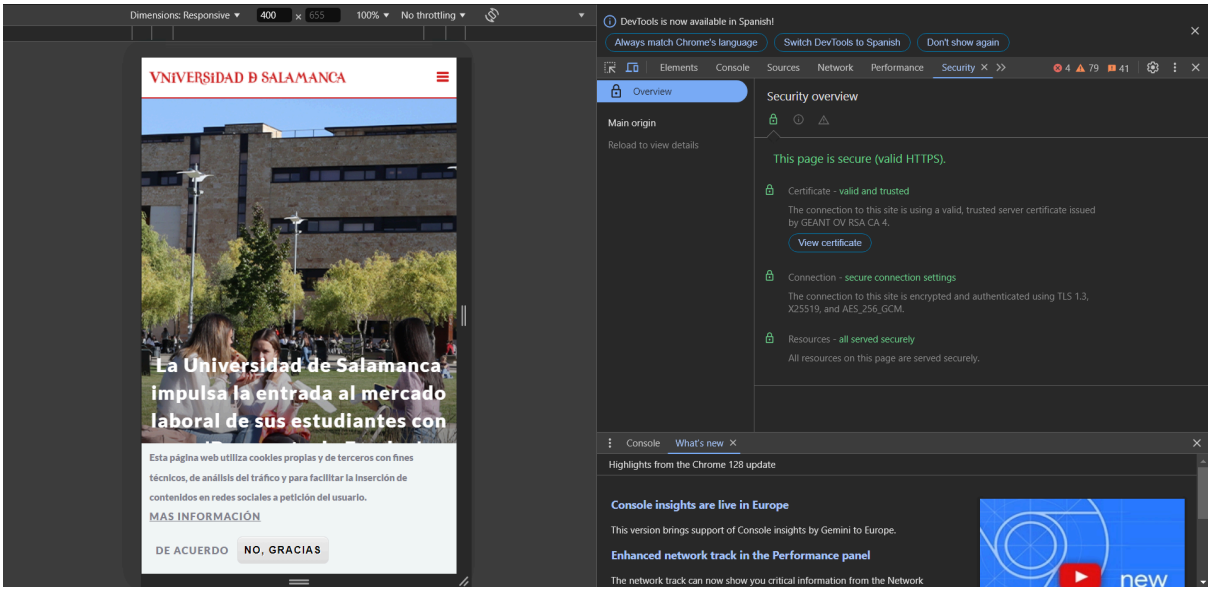
La razón de esto es que los algoritmos ecb y cbc cifran por bloques, es decir, que si el texto de entrada no es múltiplo del tamaño del bloque, se va a añadir información extra al final del texto para poder cifrar bloques enteros, añadiendo así tamaño efectivo al texto cifrado.

Mientras que en los modos de operación cfb y ofb, no es relevante el tamaño del texto de entrada y se pueden cifrar textos de cualquier tamaño, sin tener que ser por bloques, por lo que cifran únicamente el texto original sin tener que añadir caracteres extra y por tanto sin aumentar su tamaño efectivo.

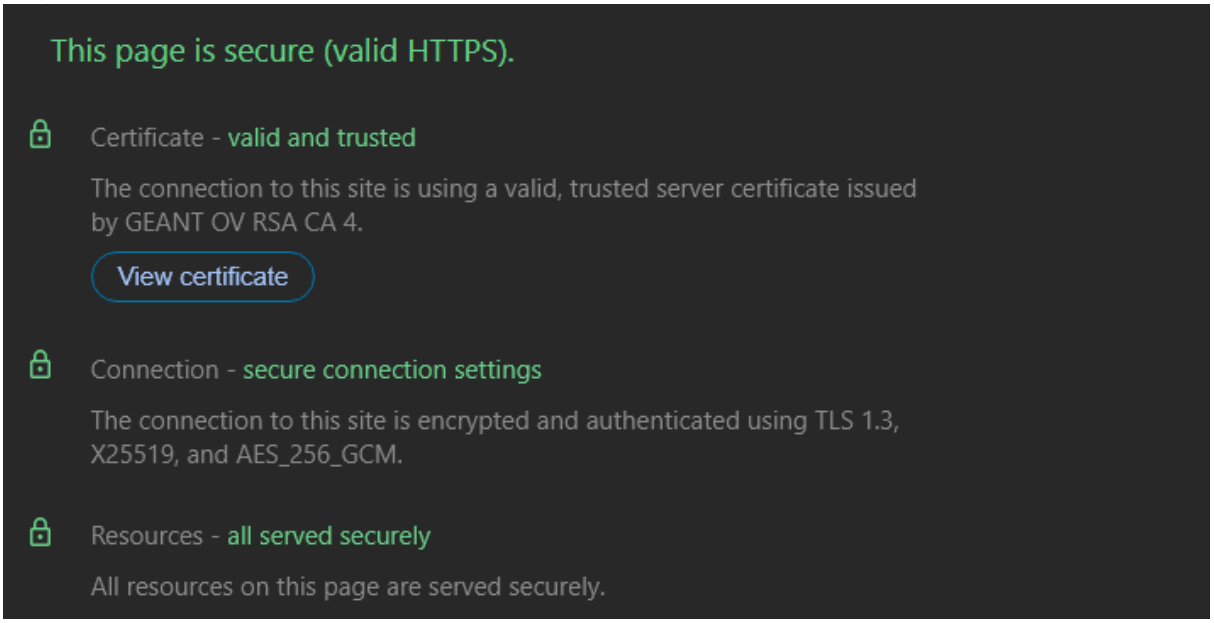
A2.- Determinar qué algoritmo de cifrado se utiliza en las conexiones HTTPs

- **Acceder a una página alojada en un servidor seguro (protocolo https) p.e. https://www.usal.es**
    - **Ver qué algoritmo simétrico se utiliza para conexión según el navegador usado (IE, Firefox, Chrome)**
- **Probar con al menos otras páginas seguras (www.google.com).**

Iniciamos nuestro navegador, en este caso Google Chrome y nos dirigimos hacia la página web. Presionamos “F12” para inspeccionarla o click derecho e inspeccionar.



Desde aquí, nos dirigiremos al apartado de Security. En la parte de Connection podremos observar el algoritmo simétrico de cifrado que se está utilizando.



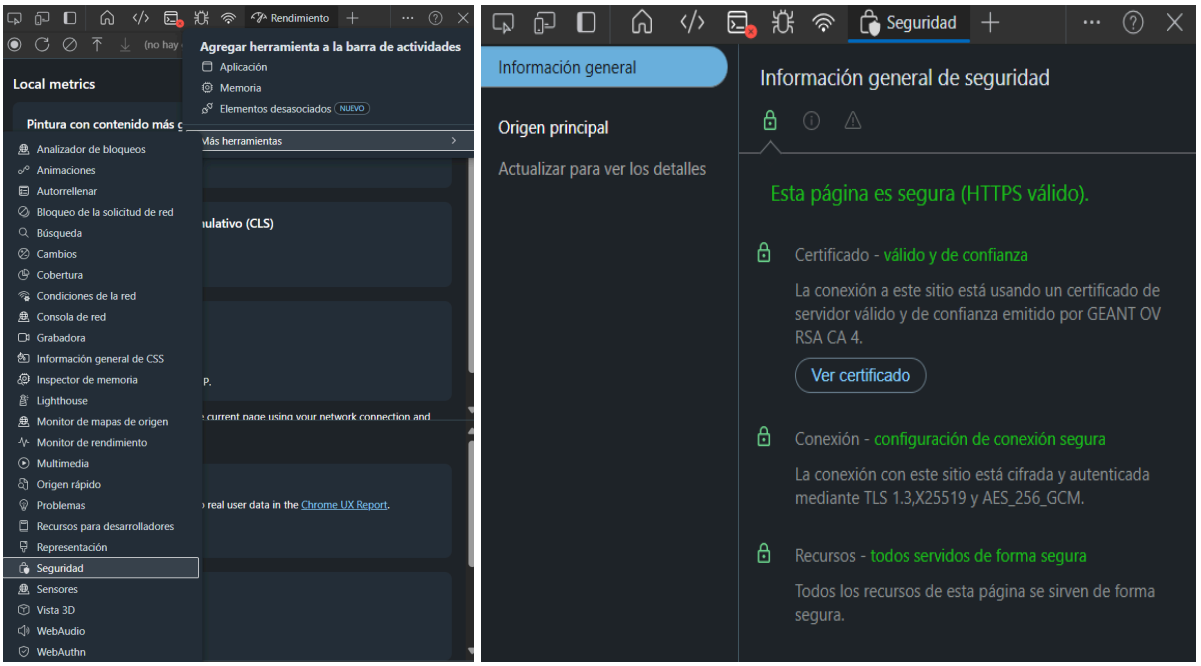
En el apartado de Connection podremos ver todos los datos del algoritmo de cifrado.

- Primeramente veremos el protocolo utilizado: TLS 1.3.
- El método de autenticación en este caso es el X25519.
- El algoritmo es AES 256 GCM. Un algoritmo, por tanto, de cifrado simétrico AES, en clave 256 y con el modo de operación Galois Counter Mode.

Ahora realizaremos el mismo procedimiento desde el navegador Edge. Navegamos hacia www.usal.es y presionamos “F12”.



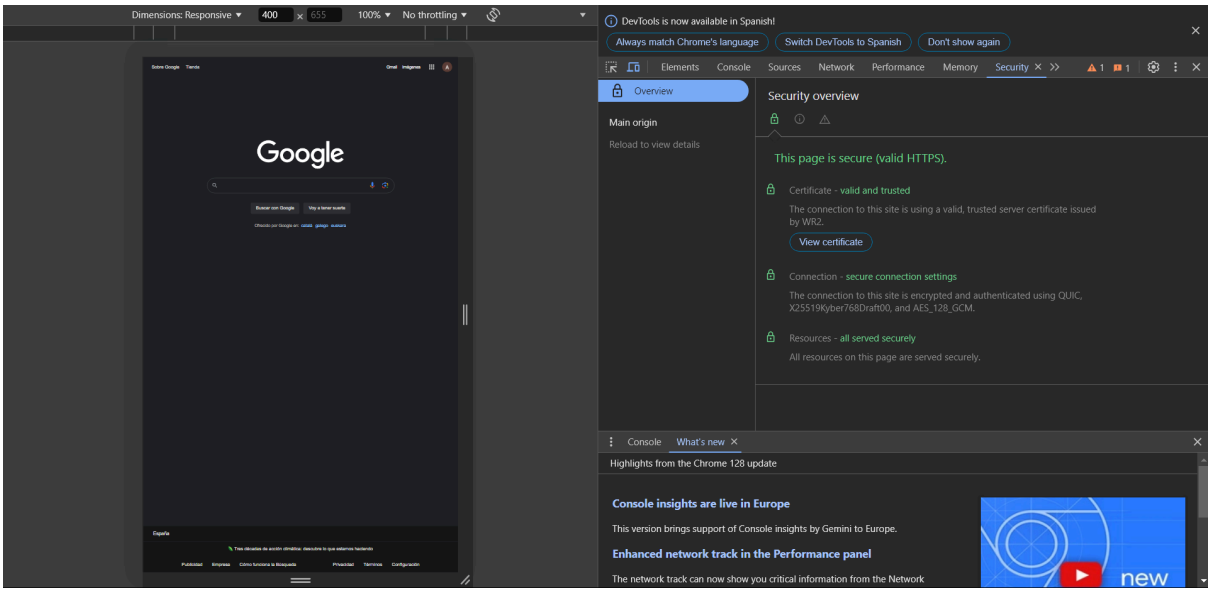
Tras ello buscamos el apartado de “Más herramientas” y seleccionamos la de seguridad.



Podremos observar en el apartado Conexión el protocolo, el método de autenticación y el algoritmo de cifrado usado, que en este caso, concuerda con el utilizado en Chrome.

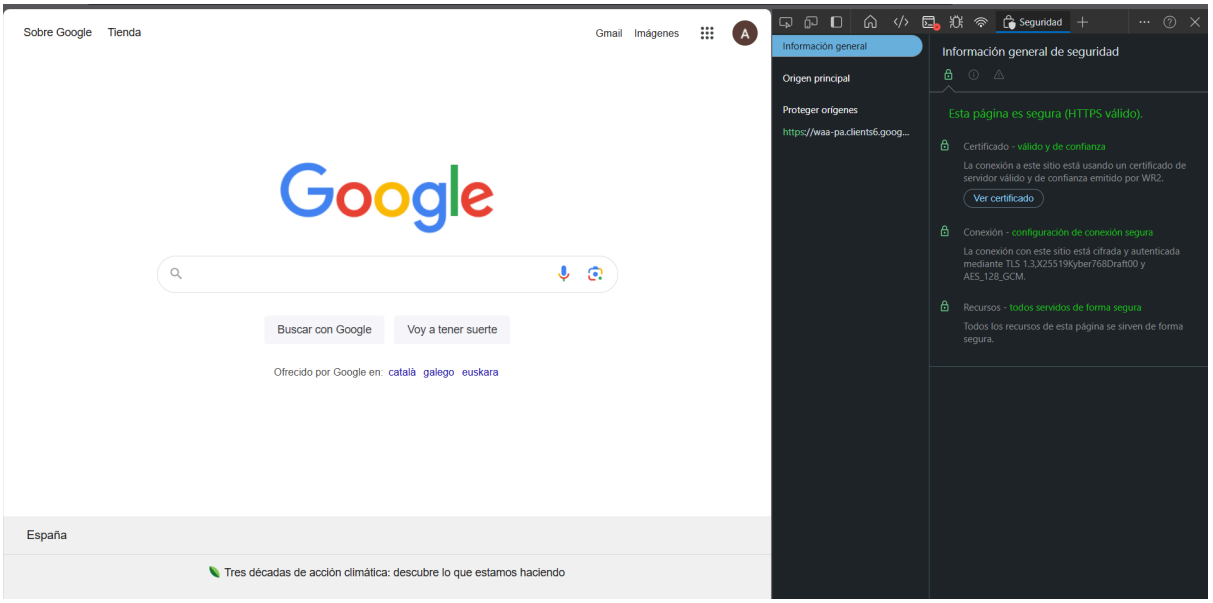
Vamos a probar con otra página distinta, en este caso, www.chrome.com.

Desde Chrome realizamos los mismos pasos y podremos ver lo siguiente:



Podemos observar que esta página utiliza el protocolo QUIC, con un método de autenticación X25519Kyber768Draft00 y un algoritmo de encriptación AES 128 GCM.

Probamos ahora desde Edge:



Podemos observar que utilizan los mismos parámetros de encriptación y autenticación que los vistos desde Chrome.

Parte B: Cifrado asimétrico

B1.- Cifrar un mensaje usando gnuPG

Con gnuPG, trabajando por parejas intercambiar ficheros cifrados usando un mecanismo de clave pública. Probar con mensajes de distintos tamaños, inferiores y superiores al tamaño de la clave que se esté usando.

- Usar el algoritmo (y por tanto claves) RSA para cifra y firma

Usamos gpg –list-keys para comprobar que no tenemos ninguna clave creada:

```
C:\Users\unaig>gpg --list-keys

C:\Users\unaig>gpg
```

Generamos las claves utilizando el comando gpg –gen-key ó gpg –full-generate-key para mostrar el diálogo completo de la generación de estas claves:

Clave generada por Alberto:

```
C:\Users\alber>gpg --full-generate-key
gpg (GnuPG) 2.4.5; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Por favor seleccione tipo de clave deseado:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sólo firmar)
  (14) Existing key from card
Su elección: 1
las claves RSA pueden tener entre 1024 y 4096 bits de longitud.
¿De qué tamaño quiere la clave? (3072) 2048
El tamaño requerido es de 2048 bits
Por favor, especifique el período de validez de la clave.
         0 = la clave nunca caduca
      <n>  = la clave caduca en n días
      <n>w = la clave caduca en n semanas
      <n>m = la clave caduca en n meses
      <n>y = la clave caduca en n años
¿Validez de la clave (0)? 365
La clave caduca 09/24/25 12:17:01 Hora de verano romance
¿Es correcto? (s/n) s
```

Introducimos los datos necesarios para la generación de la clave solicitados por gpg y asignamos una contraseña para la generación de esta clave.

```
GnuPG debe construir un ID de usuario para identificar su clave.

Nombre y apellidos: Alberto García Martín
Dirección de correo electrónico: beetogm@usal.es
Comentario: Comentario adicional
Está usando el juego de caracteres 'utf-8'.
Ha seleccionado este ID de usuario:
    "Alberto García Martín (Comentario adicional) <beetogm@usal.es>"

¿Cambia (N)ombre, (C)omentario, (D)irección o (V)ale/(S)alir? V
```

Finalmente la clave estaría correctamente generada.

```
Es necesario generar muchos bytes aleatorios. Es una buena idea realizar
alguna otra tarea (trabajar en otra ventana/consola, mover el ratón, usar
la red y los discos) durante la generación de números primos. Esto da al
generador de números aleatorios mayor oportunidad de recoger suficiente
entropía.
Es necesario generar muchos bytes aleatorios. Es una buena idea realizar
alguna otra tarea (trabajar en otra ventana/consola, mover el ratón, usar
la red y los discos) durante la generación de números primos. Esto da al
generador de números aleatorios mayor oportunidad de recoger suficiente
entropía.
gpg: certificado de revocación guardado como 'C:\\Users\\alber\\AppData\\Roaming\\gnupg\\openpgp-revocs.d\\616B53AEFCA5B989A81C222261838F5AFE1A46EB.rev'
claves pública y secreta creadas y firmadas.

pub   rsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      616B53AEFCA5B989A81C222261838F5AFE1A46EB
uid                      Alberto García Martín (Comentario adicional) <beetogm@usal.es>
sub   rsa2048 2024-09-24 [E] [caduca: 2025-09-24]
```

Clave generada por Unai:

```
C:\Users\unaig>gpg --full-generate-key
gpg (GnuPG) 2.4.5; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Por favor seleccione tipo de clave deseado:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sólo firmar)
  (14) Existing key from card
Su elección: 1
las claves RSA pueden tener entre 1024 y 4096 bits de longitud.
¿De qué tamaño quiere la clave? (3072) 2048
El tamaño requerido es de 2048 bits
Por favor, especifique el período de validez de la clave.
         0 = la clave nunca caduca
      <n>  = la clave caduca en n días
      <n>w = la clave caduca en n semanas
      <n>m = la clave caduca en n meses
      <n>y = la clave caduca en n años
¿Validez de la clave (0)? 365
La clave caduca 09/24/25 12:17:23 Hora de verano romance
¿Es correcto? (s/n)
```

```
GnuPG debe construir un ID de usuario para identificar su clave.

Nombre y apellidos: Unai Garcia Blanco
Dirección de correo electrónico: unaigb@usal.es
Comentario: SSI PA2
Ha seleccionado este ID de usuario:
    "Unai Garcia Blanco (SSI PA2) <unaigb@usal.es>"
```

```
gpg: creado el directorio 'C:\\Users\\unaig\\AppData\\Roaming\\gnupg\\openpgp-revocs.d'
gpg: certificado de revocación guardado como 'C:\\Users\\unaig\\AppData\\Roaming\\gnupg\\openpgp-revocs.d\\9CD503A43C84E
46EF2ADAF4426B713F00CF53A53.rev'
claves pública y secreta creadas y firmadas.

pub   rsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      9CD503A43C84E46EF2ADAF4426B713F00CF53A53
uid                    Unai Garcia Blanco (SSI PA2) <unaigb@usal.es>
sub   rsa2048 2024-09-24 [E] [caduca: 2025-09-24]
```

Usando gpg –list-keys:

```
C:\Users\alber>gpg --list-keys
gpg: comprobando base de datos de confianza
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: nivel: 0  validez:   1  firmada:   0  confianza: 0-, 0q, 0n, 0m, 0f, 1u
gpg: siguiente comprobación de base de datos de confianza el: 2025-09-24
[keyboxd]
---------
pub   rsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      616B53AEFCA5B989A81C222261838F5AFE1A46EB
uid        [  absoluta ] Alberto García Martín (Comentario adicional) <beetogm@usal.es>
sub   rsa2048 2024-09-24 [E] [caduca: 2025-09-24]


C:\Users\alber>
```

```
C:\Users\unaig>gpg --list-keys
gpg: comprobando base de datos de confianza
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: nivel: 0  validez:   1  firmada:   0  confianza: 0-, 0q, 0n, 0m, 0f, 1u
gpg: siguiente comprobación de base de datos de confianza el: 2025-09-24
[keyboxd]
---------
pub   rsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      9CD503A43C84E46EF2ADAF4426B713F00CF53A53
uid        [  absoluta ] Unai Garcia Blanco (SSI PA2) <unaigb@usal.es>
sub   rsa2048 2024-09-24 [E] [caduca: 2025-09-24]
```

A continuación realizaremos el cifrado/descifrado del archivo. Para empezar, compartiremos las claves. Se realizará el cifrado con la clave pública y el descifrado con la clave privada del destinatario.

Para exportar la clave utilizaremos el comando gpg –export –output C:\Users\alber\Desktop\rsa-pub.bin

```
C:\Users\alber>gpg --export --output C:\Users\alber\Desktop\rsa-pub.bin

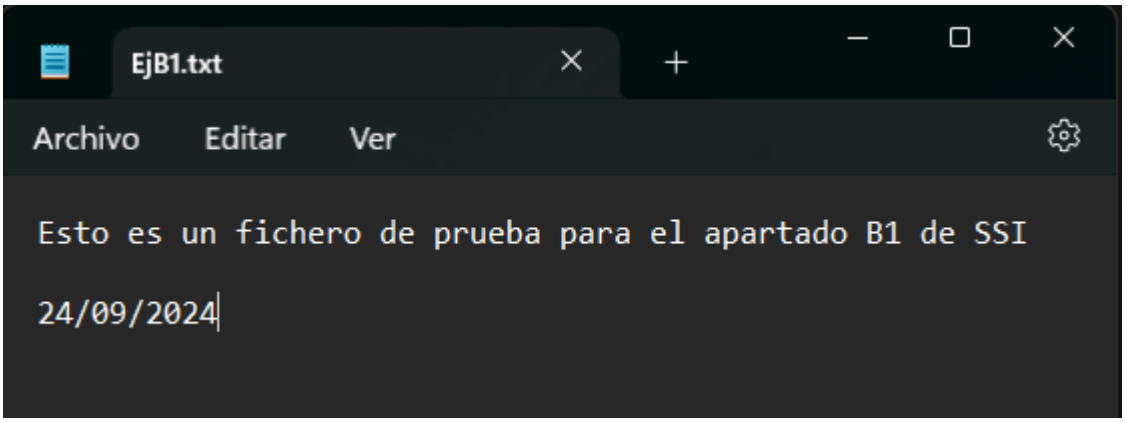
C:\Users\alber>
```

Quien vaya a cifrar y compartir el fichero importará la clave pública:

gpg --import C:\Users\unaig\Desktop\rsa-pub.bin

```
C:\Users\unaig\Desktop>gpg --import C:\Users\unaig\Desktop\rsa-pub.bin
gpg: clave 61838F5AFE1A46EB: clave pública "Alberto García Martín (Comentario adicional) <beetogm@usal.es>" importada
gpg: Cantidad total procesada: 1
gpg:               importadas: 1
```

Después se creará el fichero y se encriptará de la siguiente manera:



gpg --encrypt -r beetogm@usal.es C:\Users\unaig\Desktop\EjB1.txt

```
C:\Users\unaig\Desktop>gpg --import C:\Users\unaig\Desktop\rsa-pub.bin
gpg: clave 61838F5AFE1A46EB: clave pública "Alberto García Martín (Comentario adicional)
 <beetogm@usal.es>" importada
gpg: Cantidad total procesada: 1
gpg:               importadas: 1

C:\Users\unaig\Desktop>gpg --encrypt -r beetogm@usal.es C:\Users\unaig\Desktop\EjB1.txt
gpg: 47B82D41FDA133BC: No hay seguridad de que esta clave pertenezca realmente
al usuario que se nombra

sub  rsa2048/47B82D41FDA133BC 2024-09-24 Alberto García Martín (Comentario adicional) <b
eetogm@usal.es>
 Huella clave primaria: 616B 53AE FCA5 B989 A81C  2222 6183 8F5A FE1A 46EB
      Huella de subclave: C4AD EED6 76A6 727F 5CB9  9DE0 47B8 2D41 FDA1 33BC

No es seguro que la clave pertenezca a la persona que se nombra en el
identificador de usuario. Si *realmente* sabe lo que está haciendo,
puede contestar sí a la siguiente pregunta.

¿Usar esta clave de todas formas? (s/N) s
```

En el propio texto, tras cifrar con la clave de otra persona nos indica que la clave podría no ser segura, por esto, convendría firmar la clave antes de compartirla para garantizar su seguridad.

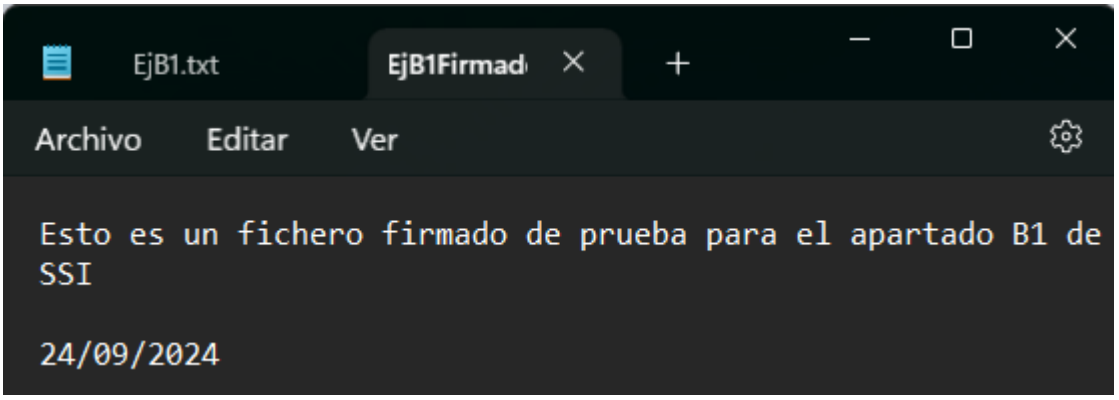
Para descifrar el fichero cifrado se utiliza:

gpg --decrypt -r beetogm@usal.es C:\Users\alber\Desktop\EjB1.txt.gpg

```
C:\Users\alber>gpg --decrypt -r beetogm@usal.es C:\Users\alber\Desktop\EjB1.txt.gpg
gpg: encrypted with rsa2048 key, ID 47B82D41FDA133BC, created 2024-09-24
      "Alberto García Martín (Comentario adicional) <beetogm@usal.es>"
Esto es un fichero de prueba para el apartado B1 de SSI

24/09/2024
C:\Users\alber>
```

Podemos observar que el texto es idéntico al cifrado anteriormente.



A continuación realizaremos una encriptación con firma con los comandos:

Se encripta: gpg --encrypt -r Comentario C:\Users\alber\Desktop\EjB1Firmado.txt

Se firma: gpg --sign C:\Users\alber\Desktop\EjB1Firmado.txt.gpg

```
C:\Users\alber>gpg --encrypt -r Comentario C:\Users\alber\Desktop\EjB1Firmado.txt

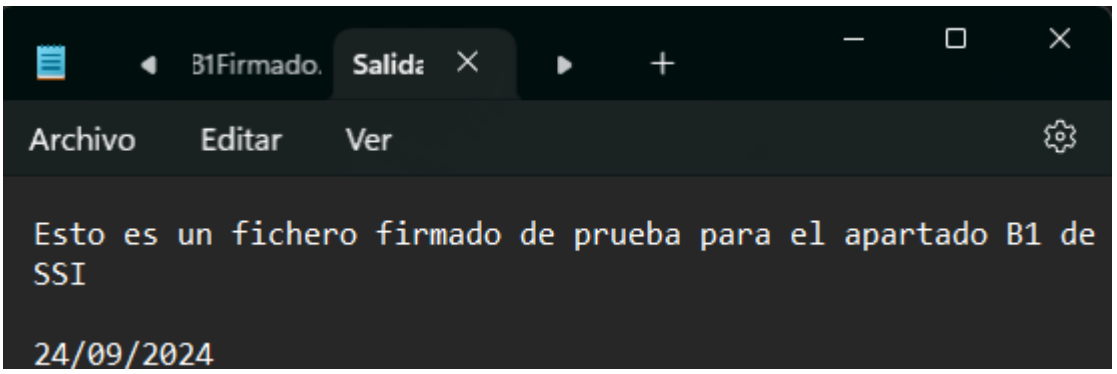
C:\Users\alber>gpg --sign C:\Users\alber\Desktop\EjB1Firmado.txt.gpg
El fichero 'C:\Users\alber\Desktop\EjB1Firmado.txt.gpg.gpg' ya existe. ¿Sobreescribir? (s/N) s

C:\Users\alber>
```

Posteriormente, verificaremos que la firma es válida desde el otro equipo con el comando gpg --verify -r beetogm@usal.es C:\Users\unaig\Desktop\EjB1firmado.txt.gpg.gpg > C:\Users\unaig\Desktop\SalidaB1.txt

```
C:\Users\unaig\Desktop>gpg --verify -r beetogm@usal.es C:\Users\unaig\Desktop\EjB1firmad
o.txt.gpg.gpg > C:\Users\unaig\Desktop\SalidaB1.txt
gpg: Firmado el 09/24/24 12:45:24 Hora de verano romance
gpg:                usando RSA clave 616B53AEFCA5B989A81C222261838F5AFE1A46EB
gpg: Firma correcta de "Alberto García Martín (Comentario adicional) <beetogm@usal.es>"
[desconocido]
gpg: ATENCIÓN: ¡Esta clave no está certificada por una firma de confianza!
gpg:          No hay indicios de que la firma pertenezca al propietario.
Huellas dactilares de la clave primaria: 616B 53AE FCA5 B989 A81C  2222 6183 8F5A FE1A 4
6EB
```

Y el fichero resultante es el siguiente:



- Usar el algoritmo El Gamal para cifrar y DSA para firmar

Realizamos los mismos pasos que para el anterior apartado, seleccionando en este caso la opción 2 de tipo de clave deseado.

```
C:\Users\alber>gpg --full-generate-key
gpg (GnuPG) 2.4.5; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

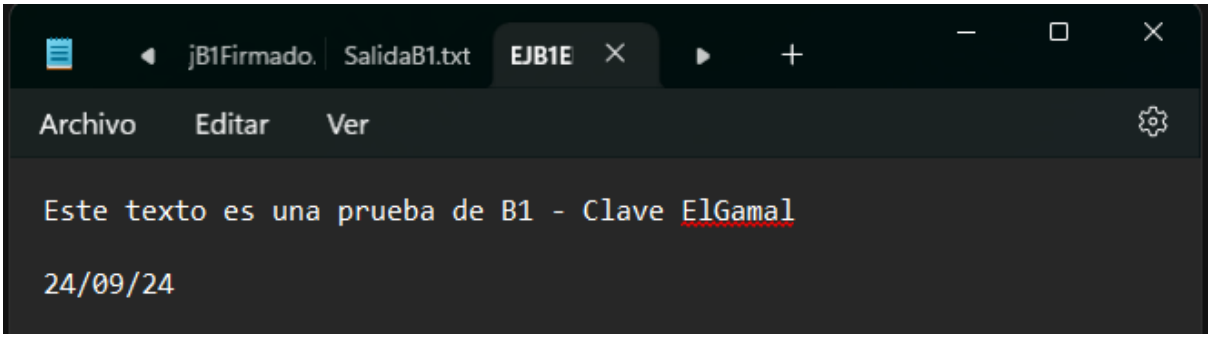
Por favor seleccione tipo de clave deseado:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sólo firmar)
  (14) Existing key from card
Su elección: 2
las claves DSA pueden tener entre 1024 y 3072 bits de longitud.
¿De qué tamaño quiere la clave? (2048)
El tamaño requerido es de 2048 bits
Por favor, especifique el período de validez de la clave.
         0 = la clave nunca caduca
      <n>  = la clave caduca en n días
      <n>w = la clave caduca en n semanas
      <n>m = la clave caduca en n meses
      <n>y = la clave caduca en n años
¿Validez de la clave (0)? 365
La clave caduca 09/24/25 12:56:30 Hora de verano romance
¿Es correcto? (s/n) s
```

Comprobamos que la clave ha sido correctamente generada.

```
C:\Users\alber>gpg --list-keys
gpg: comprobando base de datos de confianza
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: nivel: 0  validez:   2  firmada:   0  confianza: 0-, 0q, 0n, 0m, 0f, 2u
gpg: siguiente comprobación de base de datos de confianza el: 2025-09-24
[keyboxd]
---------
pub   rsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      616B53AEFCA5B989A81C222261838F5AFE1A46EB
uid        [  absoluta ] Alberto García Martín (Comentario adicional) <beetogm@usal.es>
sub   rsa2048 2024-09-24 [E] [caduca: 2025-09-24]

pub   dsa2048 2024-09-24 [SC] [caduca: 2025-09-24]
      CE049F144A592C656D3BF2391C17475D3F976259
uid        [  absoluta ] Alberto García Martín (ComentarioB2) <beetogm@usal.es>
sub   elg2048 2024-09-24 [E] [caduca: 2025-09-24]
```

Se crear fichero a cifrar:



Para ello, como anteriormente, exportaremos e importaremos la clave pública:

```
C:\Users\alber>gpg --export --output C:\Users\alber\Desktop\dsa-pub.bin

C:\Users\alber>
```

```
C:\Users\unaig\Desktop>gpg --import C:\Users\unaig\Desktop\dsa-pub.bin
gpg: clave 61838F5AFE1A46EB: "Alberto García Martín (Comentario adicional) <beetogm@usal.es>"
 sin cambios
gpg: clave 1C17475D3F976259: clave pública "Alberto García Martín (ComentarioB2) <beetogm@usa
l.es>" importada
gpg: Cantidad total procesada: 2
gpg:               importadas: 1
gpg:             sin cambios: 1
```

Y cifraremos el fichero:

```
C:\Users\unaig\Desktop>gpg --encrypt -r beetogm@usal.es C:\Users\unaig\Desktop\EJB1ElGamal.tx
t
gpg: 24FA74157F0CC709: No hay seguridad de que esta clave pertenezca realmente
al usuario que se nombra

sub  elg2048/24FA74157F0CC709 2024-09-24 Alberto García Martín (ComentarioB2) <beetogm@usal.e
s>
 Huella clave primaria: CE04 9F14 4A59 2C65 6D3B  F239 1C17 475D 3F97 6259
      Huella de subclave: 5593 5F83 D54D BE0C D9A0  80F4 24FA 7415 7F0C C709

No es seguro que la clave pertenezca a la persona que se nombra en el
identificador de usuario. Si *realmente* sabe lo que está haciendo,
puede contestar sí a la siguiente pregunta.

¿Usar esta clave de todas formas? (s/N) s

C:\Users\unaig\Desktop>
```

Ahora, desencriptamos desde el otro equipo con el comando:

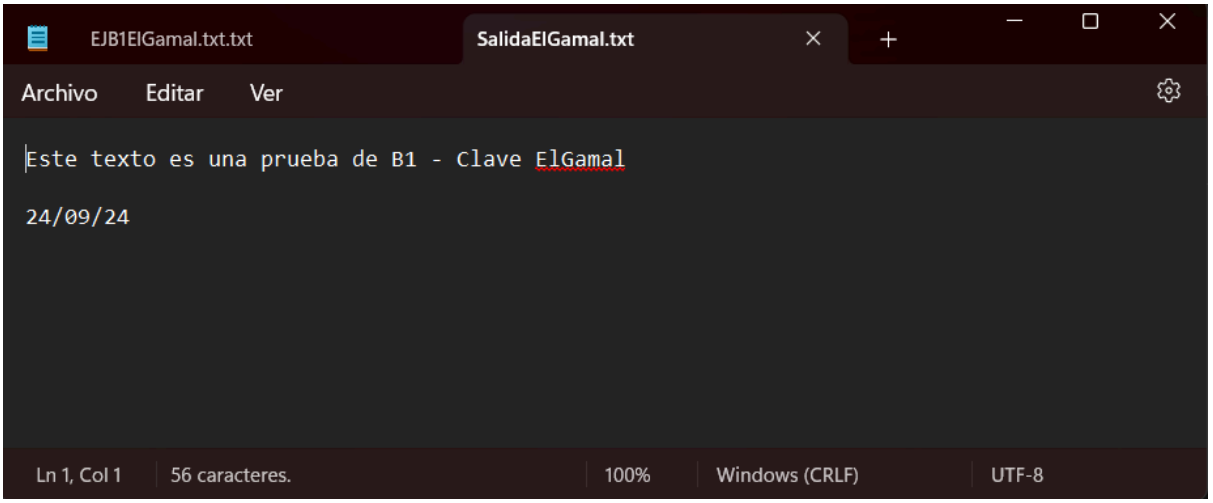
gpg --decrypt -r beetogm@usal.es C:\Users\alber\Desktop\EJB1ElGamal.txt.gpg > C:\Users\alber\Desktop\SalidaElGamal.txt

```
C:\Users\alber>gpg --export --output C:\Users\alber\Desktop\dsa-pub.bin

C:\Users\alber>gpg --decrypt -r beetogm@usal.es C:\Users\alber\Desktop\EJB1ElGamal.txt.gpg > C:\Users\alber\Desktop\SalidaElGamal.txt
gpg: encrypted with elg2048 key, ID 24FA74157F0CC709, created 2024-09-24
      "Alberto García Martín (ComentarioB2) <beetogm@usal.es>"

C:\Users\alber>
```

Comprobamos que el texto es el mismo:



B2.- Cifrar un mensaje usando openSSL

Trabajando por parejas intercambiar ficheros cifrados usando un mecanismo de clave pública con algoritmo RSA. pero usando la suite openSSL. Hay que tener en cuenta que el manejo de las claves pública y privada se hace directamente a través de ficheros y no de anillos de claves.

Comenzamos generando dos claves pública y privada con openssl genrsa 2048.

```
C:\Users\alber>openssl genrsa 2048
Loading 'screen' into random state - done
Generating RSA private key, 2048 bit long modulus
.....
.....+++
e is 65537 (0x10001)
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAwRJ6jJ1NKUKG70ZE15nddbDSnJ+OvffkKJTxfXirslqrO/Aw
hAfknbNuDzSzX8ZcJzdsbYPUjixQNuRe52jcdhnHUfrxqdOi16BOTf4EeppQZxpw
Spau7k+U7X0BfN3woE4vb/jCvwedRXyJuM/HhvE+1U5kjna7jWViF2X0e3oYLZ9I
QuWD6m3erZMKbr27twCdeapyCBcuIJsd+HJoXUcpk0p7Xlf67m07lFr+YbpApzrI
0CdFmYWaO6q9ubRpq0bzYb9GThR9Wbgx+wpeSXN3RoLM3HSPP2IbQN5+DJMaPWoQ
zODfLgzAk/PaNU1aBUTsqQ6j3g6S1lCaH+l08QIDAQABAoIBAQCpjGCyOzEXmqSE
W2D5NNWWFMHfaP8hNB/Yw3OOi/m7loPpDvSL/aJ2Gqv8Dnu3D6q9hZ5iTfLLJ38J
tWLiNOPlkp3HEGhIiuHmGZCqTPgB8qHI6R7TYaAdByXxqGU2EnDNVduyqPHccANc
khEfitfdWN2k1ELaGMp7URzjD1WzV+4JNQEHLdIrGryO/uEUN2zkWdvUX439Qtps
QQ1Us5I0U1iQZHO0Q4CpMfTFN3DQGhod9trRUyvn6IEQrFyOJTjqSJn20D9DYCMT
OjvHz6cpex6dUmSpalPBQ5yPhWrMU6VlJtfAWwhSQqYognzMPqeV+tCt9WGj04MZ
b2S7vUQBAoGBAOntxlOBKQihgZQ13bhVDxszEDVJu2PI3K8HyOpiGzIysLgiCLKt
IVl6kDjX2AEIPfCG5G3fYCy0N9I8p18TCIj0oTLwcab+yWc41WrQ4tsjC8XkWX43
oRwWhQp3krq6/AXPN99eOqxS4E2Bcc0KKThDfDaUjuUerXWWPWslaIjJAoGBANNJ
3nY7lctiD1gEkyiYHTiN33e1025XWf2kVdL7wSdT6bnyzOdmKu/Tbv2CT6Yh2lrW
UNYHazqCkiLXBt6RmNgIamBvsSmmH7cHod/+edxtovEdVcem9zCiiD13p5kbXX7K
hVV7zajeZg9iUqDaT9A/CIWAuOWQz5xyBoX4bkbpAoGAcyRoVD3k9qyYXV1OLyce
8mJl/fS22dzKUxTf3g5AGKs+tMN7yIbTPXnId1I2rftikXu0ACjM6/Qg7ciREYkK
0G6o0avsJSiP9dDRq2v53acYamWFb0zubibfgVFt1Z+n4Swn8jgtZRi98yhd3XC8
7Ua6YxsoYrqA4rn6rTySgZECgYAscTg61FOwwQKLOAMgIcbbs3PSwUskxXjnUh6/
TzcUp6Dc/QY+kg3r/iQ1bKiGQE6mlrfQHSVf3EES1Pnb7PIiVrrRaAzyvr+jBxb0
f5lwVRtV7h6x7RvHfhh8HWQCX3AuEDk1fuIGaCk9iSfqL6FkIy4OrsG1f907gfx/
R38iqQKBgQDNzHowDLbk0xAe4RhpkTw61Yq0NLq3BA+E7PYGwrPiIvNB6IFYqfig
PgaDSsTPTh4hkgAgve82MQwF/J/sJrS11r3Cv9AmHQeUH0UimJ45NAkh2l40paWV
S2tDf79eHVSZHPyFigXaYs/WMT5OA6xTD8XFJJQAo1AKHBkXDjdkAw==
-----END RSA PRIVATE KEY-----
```

Esta clave será de 2048 bits. Podemos comprobar al generar la clave pública el valor del exponente público (e is 65537), que será igual para todos los usuarios. Lo único que variará con respecto a las demás es la clave privada.

Vamos a generar en este caso el archivo de clave de 1024 bits que compartiremos con el otro equipo con el comando openssl genrsa -out C:\Users\alber\Desktop\EjB2RSA.priv 1024

```
C:\Users\alber>openssl genrsa -out C:\Users\alber\Desktop\EjB2RSA.priv 1024
Loading 'screen' into random state - done
Generating RSA private key, 1024 bit long modulus
.......++++++
..............................++++++
e is 65537 (0x10001)
```

En este caso no se nos mostrará la clave, pero si queremos comprobarla utilizaremos el comando openssl rsa -text C:\Users\alber\Desktop\EjB2RSA.priv

```
C:\Users\alber>openssl rsa -text -in C:\Users\alber\Desktop\EjB2RSA.priv
Private-Key: (1024 bit)
modulus:
    00:a3:56:e6:0e:b5:66:a5:81:02:75:81:29:ec:2b:
    7b:0f:8d:d6:33:1a:a1:70:85:4b:66:a2:1c:b3:a5:
    6f:1a:aa:6b:14:05:53:d2:05:ef:30:76:80:9e:fa:
    f3:09:81:3c:db:17:ab:f8:3f:d5:ef:71:42:fc:8e:
    18:55:5d:ef:02:30:8c:6b:b8:14:be:aa:72:96:4f:
    93:f0:e8:9f:7a:64:93:0f:18:fa:fa:8b:00:d6:a4:
    a8:bd:7b:40:8d:80:01:d5:fa:28:68:9a:b3:f7:0d:
    1d:12:8b:15:0c:c2:4c:f3:5a:fe:a3:45:f3:fd:06:
    1d:9e:6b:d4:e8:ca:01:0c:d7
publicExponent: 65537 (0x10001)
privateExponent:
    57:7e:37:d3:4f:26:01:a7:5e:4a:d4:fa:40:d0:75:
    a5:ba:43:2f:b2:a7:21:5f:b5:28:ef:d3:52:03:5e:
    c5:38:de:0c:a6:61:e4:d1:5a:f3:69:b6:b1:f2:18:
    77:51:eb:e8:2c:00:e9:04:40:b3:76:c6:fc:03:9d:
    53:34:f2:ce:2d:5c:dc:b8:04:78:7c:42:a7:02:e4:
    3d:0e:98:d4:5d:4b:05:9d:d8:8f:ba:87:4e:41:4e:
    45:ad:fa:c6:84:61:ce:7d:3a:cf:26:01:52:2f:a4:
    7a:65:ae:a6:5c:c0:08:6b:9e:6a:ec:47:cd:af:b0:
    84:5b:71:a1:c6:81:90:81
prime1:
    00:cf:8e:e1:37:a3:da:77:df:08:3c:12:69:c5:9b:
    41:b2:2f:5e:11:57:0c:16:11:92:3c:2e:99:f3:76:
    2b:2a:46:00:43:c2:0b:14:fa:e9:8b:8c:ee:2c:69:
    0a:dc:72:7b:52:e4:a8:b3:8b:b9:70:f2:a8:14:46:
    ed:02:71:f8:c1
prime2:
    00:c9:76:0e:05:6c:88:be:d4:8c:dc:97:a1:ac:d0:
    8e:17:0d:0e:f1:44:15:49:c7:25:5d:fe:7b:45:56:
    bc:cb:cb:a3:6e:5b:21:db:72:f1:30:a0:87:1f:58:
    23:18:e7:98:0e:1e:70:13:a1:79:22:71:31:f3:50:
    31:7e:91:13:97
exponent1:
    00:8d:3b:19:c0:b4:7b:c9:a9:37:7f:38:c0:44:37:
    da:c2:16:3d:19:eb:73:4b:e8:f4:ea:82:83:3b:08:
    a3:62:86:73:53:a7:cb:c6:a0:dd:1d:99:ed:ce:c6:
    43:1f:b1:be:7c:af:1e:a3:dd:65:77:97:86:0b:16:
    1f:db:d4:15:81
exponent2:
    58:2f:e5:53:5a:7f:41:89:a9:5f:a6:c0:8e:f5:7c:
```

```
    3d:7a:e9:df:92:fd:fe:9d:be:2d:b7:08:1f:02:29:
    02:cc:d1:6d:ad:56:d4:c9:2b:eb:2d:59:f8:ce:8c:
    0e:8a:d2:74:db:be:13:d9:cb:3c:d8:70:15:22:35:
    88:41:d9:61
coefficient:
    03:b4:e5:0b:3c:7f:71:b3:20:2d:5e:fa:dd:0b:ce:
    72:4f:06:51:6a:be:2c:09:03:5e:e8:3f:42:09:88:
    ce:e0:61:57:df:93:cd:fd:4e:fd:4e:72:a2:b2:9f:
    e8:e7:b0:25:2a:4c:fb:4d:bb:7d:2c:95:b5:95:fc:
    74:88:95:91
writing RSA key
-----BEGIN RSA PRIVATE KEY-----
MIICXAIBAAKBgQCjVuYOtWalgQJ1gSnsK3sPjdYzGqFwhUtmohyzpW8aqmsUBVPS
Be8wdoCe+vMJgTzbF6v4P9XvcUL8jhhVXe8CMIxruBS+qnKWT5Pw6J96ZJMPGPr6
iwDWpKi9e0CNgAHV+ihomrP3DR0SixUMwkzzWv6jRfP9Bh2ea9ToygEM1wIDAQAB
AoGAV343008mAadeStT6QNB1pbpDL7KnIV+1KO/TUgNexTjeDKZh5NFa82m2sfIY
d1Hr6CwA6QRAs3bG/AOdUzTyzi1c3LgEeHxCpwLkPQ6Y1F1LBZ3Yj7qHTkFORa36
xoRhzn06zyYBUi+kemWuplzACGueauxHza+whFtxocaBkIECQQDPjuE3o9p33wg8
EmnFm0GyL14RVwwWEZI8LpnzdisqRgBDwgsU+umLjO4saQrccntS5Kizi7lw8qgU
Ru0CcfjBAkEAyXYOBWyIvtSM3JehrNCOFw0O8UQVSсclXf57RVa8y8ujblsh23Lx
MKCHH1gjGOeYDh5wE6F5InEx81AxfpETlwJBAI07GcC0e8mpN384wEQ32sIWPRnr
c0vo9OqCgzsIo2KGc1Ony8ag3R2Z7c7GQx+xvnyvHqPdZXeXhgsWH9vUFYECQFgv
5VNaf0GJqV+mwI71fD166d+S/f6dvi23CB8CKQLM0W2tVtTJK+stWfjOjA6K0nTb
vhPZyzzYcBUiNYhB2WECQAO05Qs8f3GzIC1e+t0LznJPBlFqviwJA17oP0IJiM7g
YVffk839Tv1OcqKyn+jnsCUqTPtNu30slbWV/HSIlZE=
-----END RSA PRIVATE KEY-----
```

Podemos observar que el exponente público sigue siendo el mismo. Los módulos vienen expresados en hexadecimal.

A su vez se mostrará el exponente privado.

Si se pone -noout no sacará al final de estos datos la clave por pantalla.

openssl rsa -text -noout -in C:\Users\alber\Desktop\EjB2RSA.priv

Si se cambia text por modulus sacará únicamente el módulo en hexadecimal

openssl rsa -modulus -noout -in C:\Users\alber\Desktop\EjB2RSA.priv

openssl rsa -pubout -in C:\Users\alber\Desktop\EjB2RSA.priv saca lo público

openssl rsa -pubout -in C:\Users\alber\Desktop\EjB2RSA.priv -out

C:\Users\alber\Desktop\EjB2RSA.pub saca a un fichero extra lo público

openssl rsa -text -pubin -in C:\Users\alber\Desktop\EjB2RSA.pub

```
C:\Users\alber>openssl rsa -pubout -in C:\Users\alber\Desktop\EjB2RSA.priv -out C:\Users\alber\Desktop\EjB2RSA.pub
writing RSA key

C:\Users\alber>openssl rsa -text -pubin -in C:\Users\alber\Desktop\EjB2RSA.pub
Modulus (1024 bit):
    00:a3:56:e6:0e:b5:66:a5:81:02:75:81:29:ec:2b:
    7b:0f:8d:d6:33:1a:a1:70:85:4b:66:a2:1c:b3:a5:
    6f:1a:aa:6b:14:05:53:d2:05:ef:30:76:80:9e:fa:
    f3:09:81:3c:db:17:ab:f8:3f:d5:ef:71:42:fc:8e:
    18:55:5d:ef:02:30:8c:6b:b8:14:be:aa:72:96:4f:
    93:f0:e8:9f:7a:64:93:0f:18:fa:fa:8b:00:d6:a4:
    a8:bd:7b:40:8d:80:01:d5:fa:28:68:9a:b3:f7:0d:
    1d:12:8b:15:0c:c2:4c:f3:5a:fe:a3:45:f3:fd:06:
    1d:9e:6b:d4:e8:ca:01:0c:d7
Exponent: 65537 (0x10001)
writing RSA key
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCjVuYOtWalgQJ1gSnsK3sPjdYz
GqFwhUtmohyzpW8aqmsUBVPSBe8wdoCe+vMJgTzbF6v4P9XvcUL8jhhVXe8CMIxr
uBS+qnKWT5Pw6J96ZJMPGPr6iwDWpKi9e0CNgAHV+ihomrP3DR0SixUMwkzzWv6j
RfP9Bh2ea9ToygEM1wIDAQAB
-----END PUBLIC KEY-----
```

Con rsautl se puede firmar, verificar, cifrar y descifrar con el algoritmo rsa

-sign

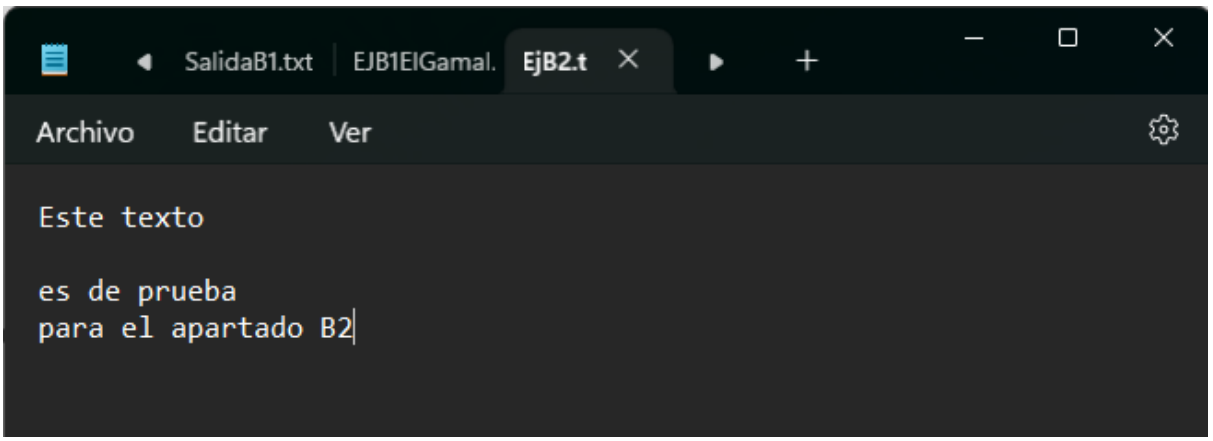
-verify

-encrypt

-decrypt

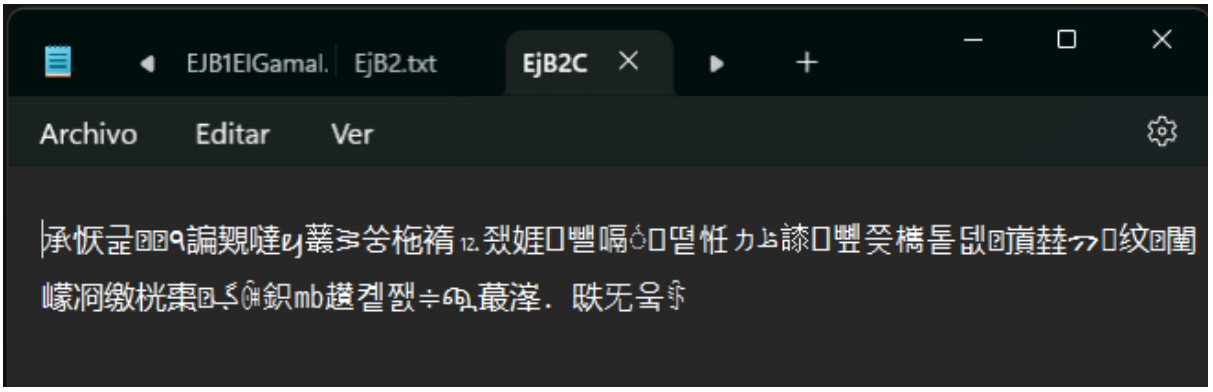
Unai creará un fichero de texto y lo encriptará usando la clave pública de Alberto.

Contenido del fichero:



Se encriptará usando la clave pública de Alberto:

rsautl -in C:\Users\unaig\Desktop\EjB2.txt -pubin -inkey C:\Users\unaig\Desktop\EjB2RSA.pub -encrypt -out C:\Users\unaig\Desktop\EjB2Cifrado.rsa




```
C:\Users\unaig>openssl
OpenSSL> rsautl -in C:\Users\unaig\Desktop\EjB2.txt -pubin -inkey C:\Users\unaig\Desktop\EjB2RSA.pub -encrypt -out C:\Us
ers\unaig\Desktop\EjB2Cifrado.rsa
Loading 'screen' into random state - done
OpenSSL>
```

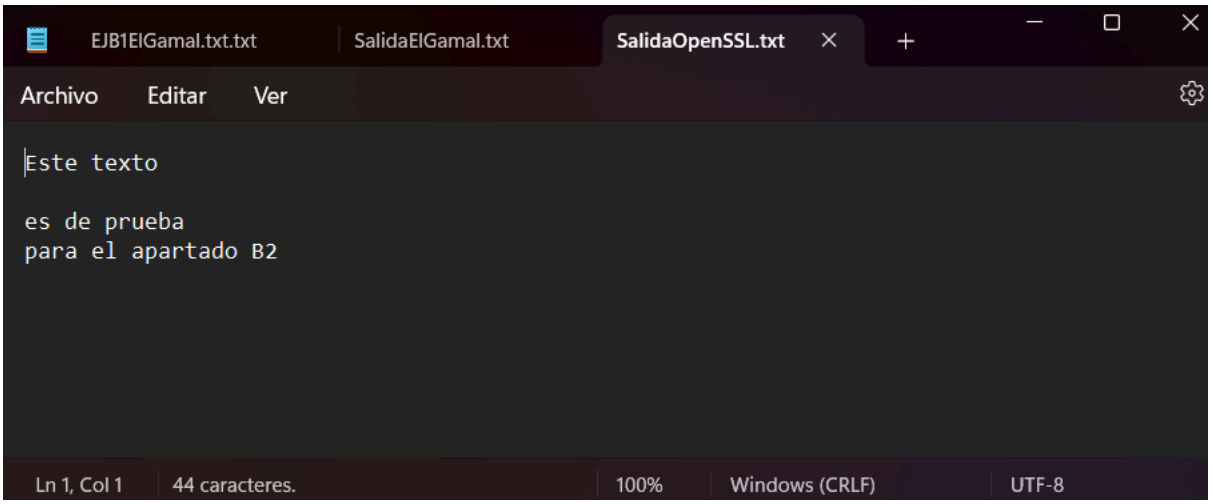
Finalmente desde el otro equipo desciframos el archivo recibido con el comando

openssl rsautl-in C:\Users\alber\Desktop\EjB2Cifrado.rsa -inkey C:\Users\alber\Desktop\EjB2RSA.priv -decrypt > C:\Users\alber\Desktop\SalidaOpenSSL.txt

```
C:\Users\alber>openssl rsautl -in C:\Users\alber\Desktop\EjB2Cifrado.rsa -inkey C:\Users\alber\Desktop\EjB2RSA.priv -dec
rypt > C:\Users\alber\Desktop\SalidaOpenSSL.txt
Loading 'screen' into random state - done

C:\Users\alber>
```

Y el texto desencriptado es el mismo como podemos comprobar:



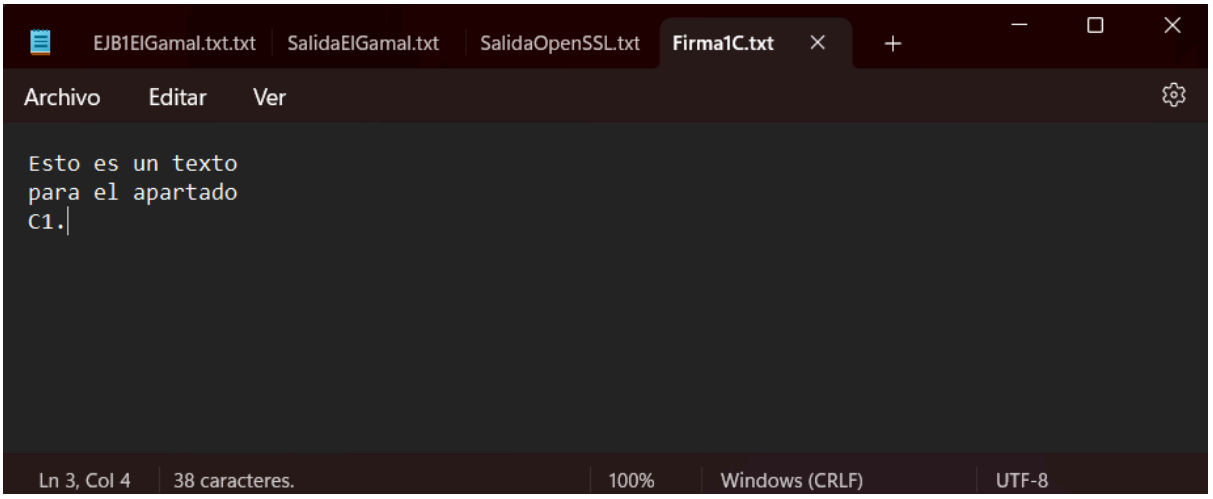
Parte C: Generar firma de mensajes

C1.- Generar una firma (separada) de un mensaje usando gnuPG

Con las parejas de claves generadas en el apartado B1 se trata de (a) generar una firma digital (solo la firma) de un mensaje. La firma tiene que estar en ASCII.

Asimismo una vez obtenida, (b) se validará dicha firma.

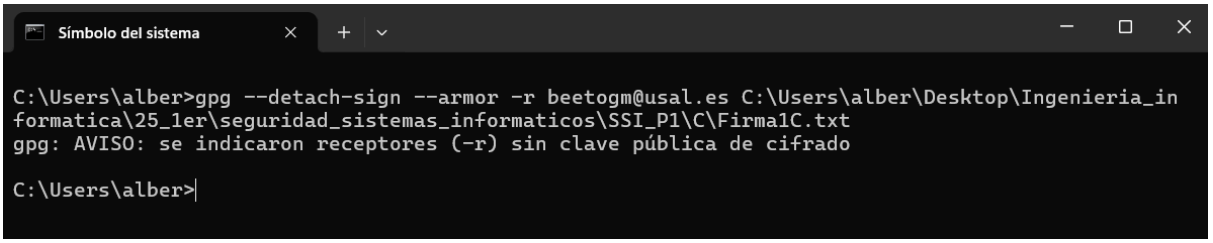
Para empezar, crearemos un fichero de texto para firmar.



Posteriormente realizaremos la firma utilizando el comando

gpg --detach-sign --armor -r beetogm@usal.es C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma1C.txt

Nos pedirá introducir la clave y se nos creará un archivo .asc del texto.



Usa el compendio SHA256 como función de hashing.

Para comprobar la firma en el otro equipo se usará verify

gpg --verify C:\Users\unaig\Downloads\Firma1C.txt.asc

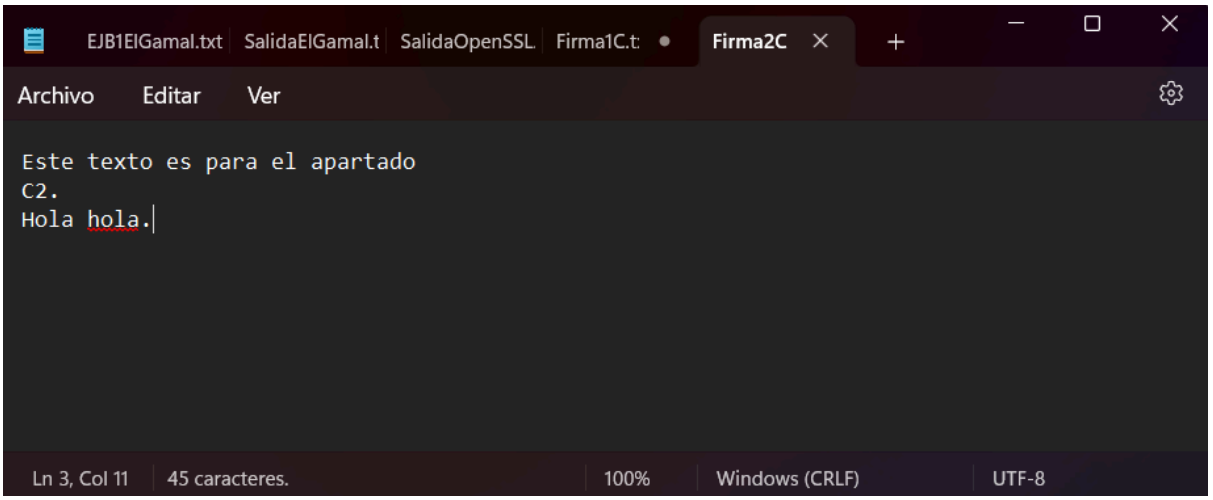
```
C:\Users\unaig>gpg --verify C:\Users\unaig\Downloads\Firma1C.txt.asc
gpg: asumiendo que los datos firmados están en 'C:\\Users\\unaig\\Downloads\\Firma1C.txt'
gpg: Firmado el 10/03/24 19:45:49 Hora de verano romance
gpg:                usando RSA clave 616B53AEFCA5B989A81C222261838F5AFE1A46EB
gpg: Firma correcta de "Alberto García Martín (Comentario adicional) <beetogm@usal.es>" [desconocido]
gpg: ATENCIÓN: ¡Esta clave no está certificada por una firma de confianza!
gpg:          No hay indicios de que la firma pertenezca al propietario.
Huellas dactilares de la clave primaria: 616B 53AE FCA5 B989 A81C  2222 6183 8F5A FE1A 46EB
```

C2.- Generar una firma del compendio de un mensaje usando openSSL

Con las parejas de claves generadas en el apartado B2 se trata de (a) generar una firma digital (solo la firma) de un mensaje. Para crear la firma del mensaje, previamente se obtendrá una huella (compendio) SHA256 del mensaje. La firma tiene que estar en ASCII.

Asimismo una vez obtenida, (b) se validará dicha firma.

Se ha creado el fichero:



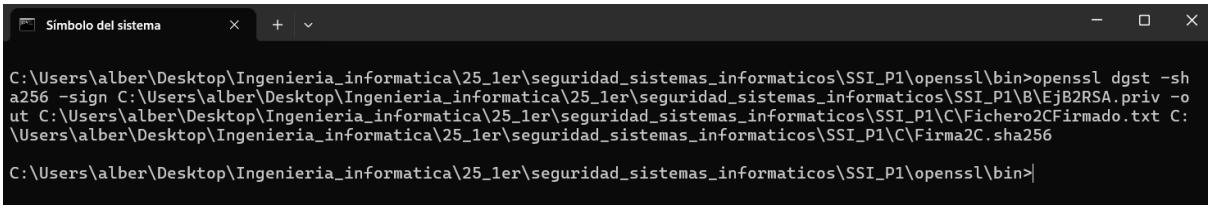
Se obtiene la huella compendio del mensaje en sha256:
openssl dgst -sha256 -out C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.sha256 C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.txt

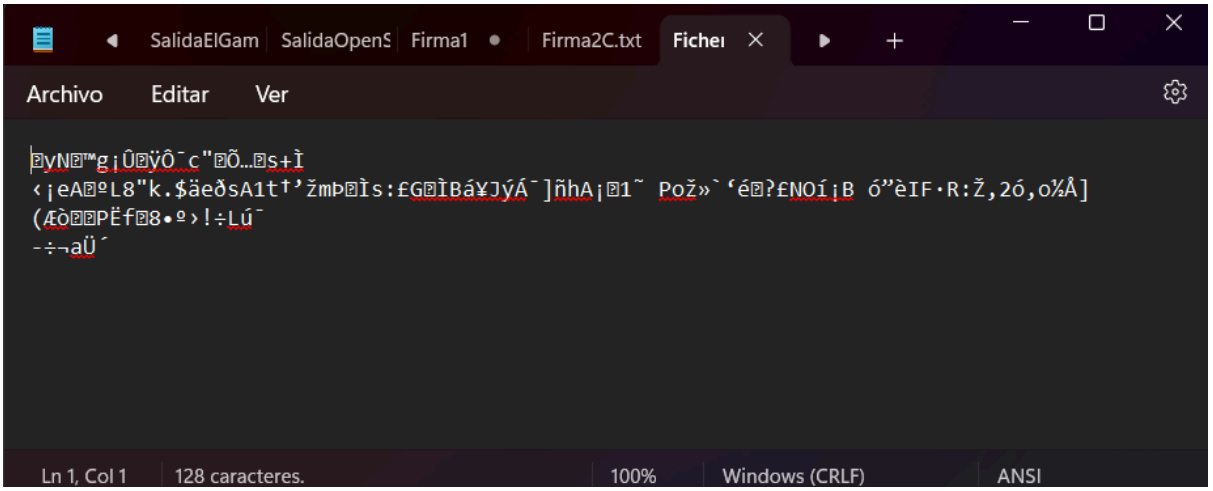
```
C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\open
ssl\bin> openssl dgst -sha256 -out C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\segurida
d_sistemas_informaticos\SSI_P1\C\Firma2C.sha256 C:\Users\alber\Desktop\Ingenieria_informatica\25
_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.txt

C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\open
ssl\bin>
```

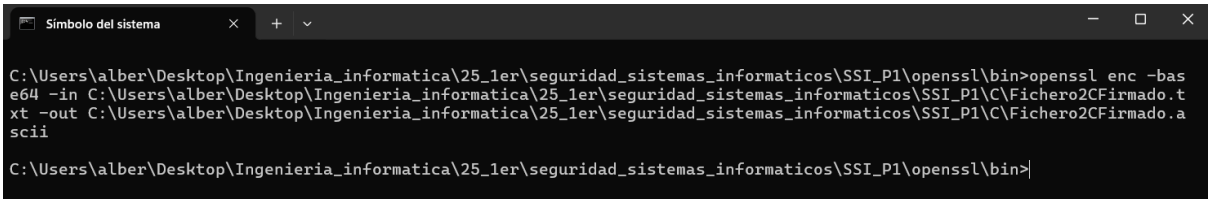
Realizaremos la firma del fichero:

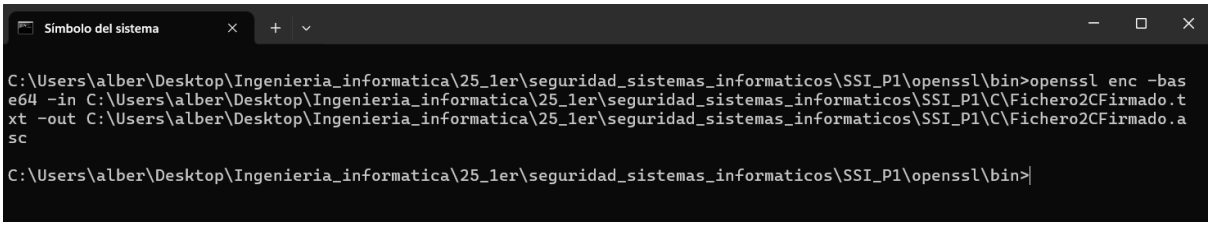
openssl dgst -sha256 -sign C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\B\EjB2RSA.priv -out C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.txt C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.sha256

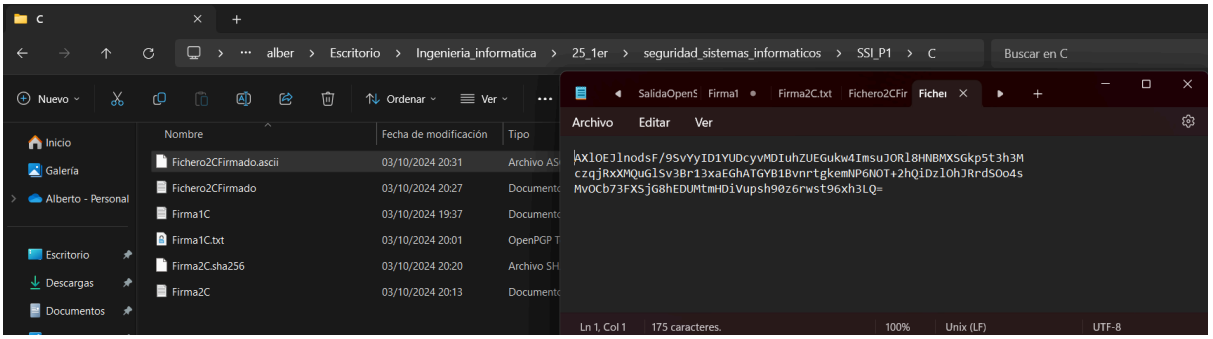




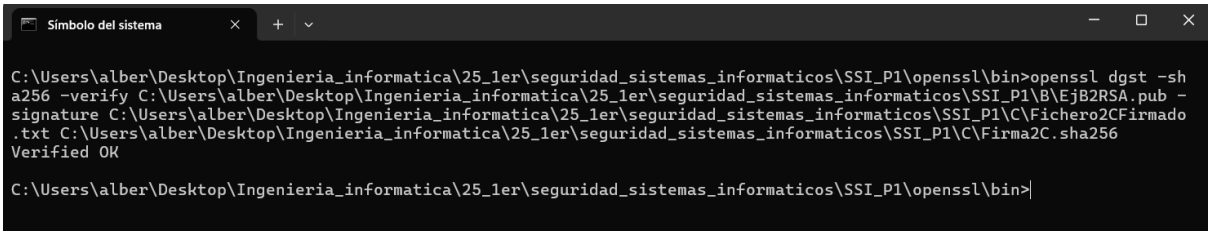
Para ver la firma en .ascii:
openssl enc -base64 -in C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.txt -out C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.asc







Se verifican las firmas:
openssl dgst -sha256 -verify C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\B\EjB2RSA.pub -signature C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.txt C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.sha256

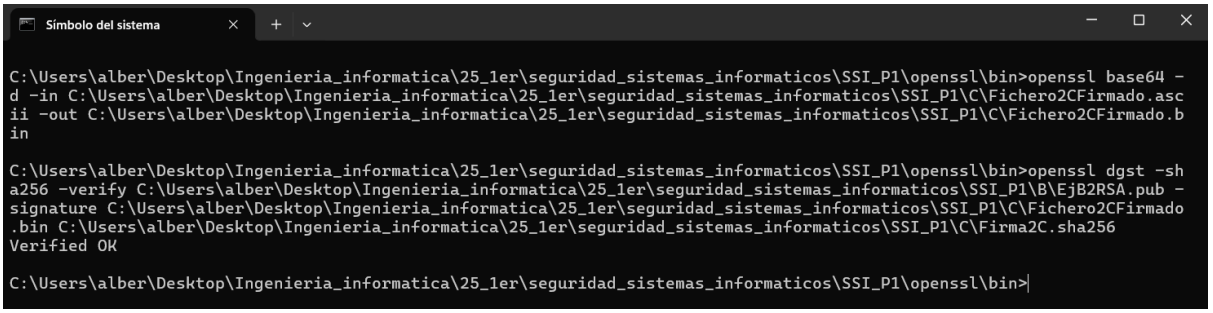


Y para la comprobación del ASCII realizamos la conversión a binario:

openssl base64 -d -in C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.ascii -out C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.bin

Y finalmente verificamos la firma binaria:

openssl dgst -sha256 -verify C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\B\EjB2RSA.pub -signature C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Fichero2CFirmado.bin C:\Users\alber\Desktop\Ingenieria_informatica\25_1er\seguridad_sistemas_informaticos\SSI_P1\C\Firma2C.sha256



Como podemos comprobar, la verificación es correcta.